

ATITUS EDUCAÇÃO
ESCOLA POLITÉCNICA
GRADUAÇÃO EM ENGENHARIA ELÉTRICA

WILLIAM SLEMAN

DESENVOLVIMENTO DE UM DISPOSITIVO PARA
AQUISIÇÃO DE DADOS COM O MÓDULO NINA-B306

PASSO FUNDO

2025

WILLIAM SLEMAN

**DESENVOLVIMENTO DE UM DISPOSITIVO PARA AQUISIÇÃO DE DADOS
COM O MÓDULO NINA-B306**

Trabalho de Conclusão de Curso,
apresentado para obtenção do grau de Bacharel
no curso de Engenharia Elétrica, Escola
Politécnica, da ATITUS Educação.

Orientador: Prof. Me. Douglas Haupt

PASSO FUNDO

2025

WILLIAM SLEMAN

**DESENVOLVIMENTO DE UM DISPOSITIVO PARA AQUISIÇÃO DE DADOS
COM O MÓDULO NINA-B306**

Trabalho de Conclusão de Curso,
apresentado para obtenção do grau de Bacharel
no curso de Engenharia Elétrica, Escola
Politécnica, da ATITUS Educação, com Linha de
Pesquisa em Sistemas Embarcados.

Passo Fundo, 23 de junho de 2025.

BANCA EXAMINADORA

Prof. Me. Douglas Haupt - Mestrado – Atitus Educação - Orientador

Prof. Dr. Richard Thomas Lermen - Doutorado – Atitus Educação

Prof.^a Dr.^a Thaísa Leal da Silva - Doutorado – Atitus Educação

AGRADECIMENTOS

Em primeiro lugar, gostaria de expressar minha profunda gratidão aos meus pais, que sempre me ofereceram apoio incondicional e valiosos ensinamentos ao longo de minha vida. Vocês são a base em que construí minha determinação e coragem.

Agradeço especialmente a Natiéli Seibt, cuja compreensão e apoio foram fundamentais em minhas decisões, ajudando-me a encontrar meu caminho.

A todos os meus colegas de trabalho da Stara, meu sincero agradecimento pelos ensinamentos e pela paciência demonstrada ao longo da minha jornada profissional. Um reconhecimento especial vai para os Tiagos (Ribeiro e Rosado), que abriram diversas portas e oportunidades que me permitiram crescer nos estudos e na carreira que escolhi seguir. Não poderia deixar de mencionar também Kummer, Pause e Follmer, os primeiros engenheiros eletricitas seniores que tive o privilégio de conhecer e conviver. Eles são, e continuarão sendo, referências valiosas para o engenheiro que almejo me tornar.

À minha turma da faculdade, aos professores e aos amigos que fizeram parte dessa caminhada, meu muito obrigado! Um agradecimento especial ao meu orientador, Professor Me. Douglas Haupt, cuja dedicação e ensinamentos ao longo desses anos foram muito importantes para eu chegar até aqui. Sou imensamente grato por todo o tempo e suporte que me ofereceu.

Dedico um parágrafo especialmente a dois engenheiros eletricitas que guardo um carinho especial. A Matheus Wonsick, pela parceria e conselhos, mesmo que alguns deles não tenham sido seguidos. E a Lucas Azeredo, minha principal referência, que não só me ensinou muito sobre a engenharia e me ajudou a dar os primeiros passos em minha carreira, mas também continua a ajudar até hoje no desenvolvimento do meu conhecimento de forma autônoma e significativa.

Por último, agradeço a todos que, de alguma forma, contribuíram para a realização deste trabalho. Cada um de vocês faz parte dessa jornada e da realização deste projeto.

“Não creio que haja uma emoção, mais intensa para um inventor do que ver suas criações funcionando. Essa emoção faz você esquecer de comer, de dormir, de tudo.”

Nikola Tesla

RESUMO

Este trabalho explora as capacidades do módulo NINA-B306, utilizando seu *Open CPU* para demonstrar funcionalidades e aplicações em sistemas embarcados. Para isso, foi desenvolvido um dispositivo de aquisição de dados de temperatura ambiente. O estudo analisa aspectos técnicos do NINA-B306 e investiga a metodologia de arquitetura em camadas para a implementação do dispositivo. Foi desenvolvido um protótipo funcional em *protoboard*, dividindo a implementação em camadas de *hardware*, *software* e aplicação. O dispositivo realiza a aquisição de dados de temperatura em intervalos regulares, armazenando as leituras em uma memória externa e sendo possível extrair posteriormente utilizando um computador e um programa em Python. Além de demonstrar as capacidades do módulo, o trabalho propõe uma metodologia para o desenvolvimento de sistemas embarcados, evitando o uso de *frameworks* e permitindo uma compreensão mais profunda do funcionamento interno de um microcontrolador, frequentemente oculto pela abstração. Todo o código do módulo foi escrito em linguagem C, sendo o código fonte aberto, permitindo que qualquer interessado o reutilize em outras aplicações e proponha melhorias nas soluções apresentadas. A documentação está disponível em inglês para alcance de um maior número de pessoas.

Palavras-chave: Sistemas Embarcados; *Open CPU*; Aquisição de Dados; Arquitetura em Camadas; NINA-B306.

ABSTRACT

This work explores the capabilities of the NINA-B306 module, utilizing its Open CPU to demonstrate functionalities and applications in embedded systems. To achieve this, a data acquisition device for ambient temperature was developed. The study analyzes the technical aspects of the NINA-B306 and investigates the layered architecture methodology for the implementation of the device. A functional prototype was developed on a breadboard, dividing the implementation into layers of hardware, software, and application. The device performs temperature data acquisition at regular intervals, storing the readings in an external memory, which can subsequently be extracted using a computer and a Python program. In addition to demonstrating the module's capabilities, the work proposes a methodology for the development of embedded systems, avoiding the use of frameworks and allowing for a deeper understanding of the internal workings of a microcontroller, often obscured by abstraction. All the module's code was written in C language, with the source code being open, enabling any interested party to reuse it in other applications and propose improvements to the presented solutions. The documentation is available in English to reach a wider audience.

Keywords: Embedded Systems; Open CPU; Data Acquisition; Layered Architecture; NINA-B306.

LISTA DE ILUSTRAÇÕES

Figura 1: Um diagrama de blocos simplificado dos componentes dentro de um microcontrolador genérico	19
Figura 2: Princípio da arquitetura de software <i>Open CPU</i>	21
Figura 3: Camadas de código	23
Figura 4: A arquitetura do BLE	26
Figura 5: Modelo baseado em GATT	28
Figura 6: Módulo NINA-B306	30
Figura 7: Diagrama de blocos nRF52840.....	31
Figura 8: Placa Arduino Nano 33 BLE Sense Revisão 2.....	32
Figura 9: Arquitetura de sistemas embarcados baseada em camadas.....	33
Figura 10: Fluxograma das etapas do desenvolvimento do projeto	35
Figura 11: Diagrama de blocos do <i>hardware</i> do dispositivo.....	36
Figura 12: Arquitetura em camadas do dispositivo.....	37
Figura 13: Montagem do dispositivo em <i>protoboard</i>	38
Figura 14: Inicialização do periférico de GPIO	39
Figura 15: Inicialização do periférico de UART	40
Figura 16: Funções no cabeçalho do periférico de GPIO.....	40
Figura 17: Cabeçalho do módulo de temperatura	41
Figura 18: Implementação em linguagem C da função "temperature_get"	42
Figura 19: Fluxograma do código principal da camada de aplicação.....	43
Figura 20: Dispositivo operando normalmente	44
Figura 21: Dispositivo em modo proteção	45
Figura 22: Teste comando " <i>help</i> "	46
Figura 23: Teste comando para extração dos dados	46
Figura 24: Teste comando para ler temperatura em tempo real	47

Figura 25: Interface do pequeno programa em Python para extração dos dados.....	48
Figura 26: Gráfico da aquisição de temperatura em 2 horas e meia	48
Figura 27: Gráfico da aquisição de temperatura em 12 horas.....	49
Figura 28: Módulo NINA-B306 no esquemático da PCI	50
Figura 29: <i>Layout</i> da placa em visualização bidimensional.....	51
Figura 30: <i>Layout</i> da placa em visualização tridimensional	52
Figura 31: <i>Layout</i> da placa com indicações dos circuitos	53

LISTA DE ABREVIATURAS E SIGLAS

ATT	<i>Attribute protocol</i>
BLE	<i>Bluetooth Low Energy</i>
CI	Circuito Integrado
CPU	<i>Central Processing Unit</i>
FPU	<i>Float Point Unit</i>
FTDI	<i>Future Technology Devices International</i>
FW	<i>Firmware</i>
GATT	<i>Generic Attribute profile</i>
GPIO	<i>General Purpose Input-Output</i>
GPRS	<i>General Packet Radio Service</i>
GSM	<i>Global System for Mobile communication</i>
HCI	<i>Host Controller Interface</i>
HW	<i>Hardware</i>
I2C	<i>Inter-Integrated Circuit</i>
IOT	<i>Internet Of Things</i>
LED	<i>Light-Emitting Diode</i>
MCU	<i>Microcontroller Unit</i>
NOR	<i>Not OR</i>
PCI	Placa de Circuito Impresso
PHY	<i>Physical layer</i>
RAM	<i>Random Access Memory</i>
RGB	<i>Red Green Blue</i>
ROM	<i>Read-Only Memory</i>
SOC	<i>System On Chip</i>
SPI	<i>Serial Peripheral Interface</i>

SWD	<i>Serial Wire Debug</i>
TTL	<i>Transistor-Transistor Logic</i>
TWI	<i>Two-Wire Interface</i>
UART	<i>Universal Asynchronous Receiver-Transmitter</i>
USB	<i>Universal Serial Bus</i>

SUMÁRIO

1	INTRODUÇÃO	14
2	OBJETIVOS	15
2.1	OBJETIVO GERAL	15
2.2	OBJETIVOS ESPECÍFICOS	15
3	REVISÃO DE LITERATURA.....	16
3.1	SISTEMAS EMBARCADOS	16
3.1.1	Arquitetura	17
3.1.2	Microcontroladores	18
3.1.3	Periféricos e interfaces de comunicação	19
3.1.4	Open CPU	20
3.2	FIRMWARE	22
3.2.1	Drivers e código reutilizável	22
3.2.2	Bootloader	24
3.3	BLUETOOTH LOW ENERGY	25
3.3.1	Arquitetura	25
3.3.2	Modelo de dados e GATT	27
4	MATERIAIS E MÉTODOS	30
4.1	MATERIAIS UTILIZADOS	30
4.1.1	Módulo NINA-B306	30
4.1.2	Arduino Nano 33 BLE	31
4.1.3	Memória <i>flash</i> externa	32
4.1.4	Conversor serial para USB	33
4.1.5	Outros materiais	33
4.2	ESTRUTURA DO SISTEMA EMBARCADO.....	33
4.2.1	Camada de <i>hardware</i>	34
4.2.2	Camada de <i>software</i>	34
4.2.3	Camada de aplicação	34
4.3	FLUXO DE DESENVOLVIMENTO	34
5	RESULTADOS	36
5.1	ARQUITETURA DO DISPOSITIVO.....	36
5.2	IMPLEMENTAÇÃO DA ARQUITETURA EM CAMADAS	38
5.2.1	Camada de <i>hardware</i>	38

5.2.2	Camada de baixo nível	39
5.2.3	Camada de alto nível.....	41
5.2.4	Camada de aplicação	42
5.3	<i>TESTES E VALIDAÇÕES</i>	43
5.3.1	Teste de funcionamento em caso de falha	43
5.3.2	Teste dos comandos via UART	45
5.3.3	Teste de aquisição de temperatura	47
5.4	<i>PLACA DE CIRCUITO IMPRESSO</i>	49
5.4.1	Esquemático da placa	50
5.4.2	<i>Layout</i> da placa	50
6	CONCLUSÃO	54
	REFERÊNCIAS	56
	APÊNDICE A – ESQUEMÁTICO DA PLACA DE CIRCUITO IMPRESSO .	59
	APÊNDICE B – CÓDIGO FONTE EM C DA CAMADA DE APLICAÇÃO ...	63
	APÊNDICE C – CÓDIGO FONTE EM PYTHON DA FERRAMENTA DE EXTRAÇÃO DE DADOS	68
	APÊNDICE D – ACESSO AO CÓDIGO FONTE COMPLETO	71

1 INTRODUÇÃO

A evolução tecnológica nas últimas décadas tem possibilitado o desenvolvimento de sistemas embarcados com funcionalidades versáteis e acessíveis. Esses sistemas, considerados computadores dedicados, desempenham um papel fundamental em diversas aplicações do cotidiano, integrando eletrônica e *software* para a execução de tarefas específicas (Garcia, 2018).

Este trabalho tem como objetivo investigar e implementar um dispositivo para aquisição de dados que registra a temperatura ambiente, utilizando o módulo NINA-B306, que opera com *Open CPU*. Embora existam outros módulos *Open CPU*, a escolha desse se justifica por seu processador ARM Cortex-M4, diversos periféricos, como leitura analógica e pinos de uso geral, além de interfaces de comunicação como SPI, I2C e UART (U-blox, 2023). A possibilidade de utilizar uma placa Arduino para programar diretamente em linguagem C, sem depender do *framework*, também contribuiu para sua seleção, facilitando a implementação do protótipo em *protoboard*.

A abordagem adotada neste estudo inclui uma análise das capacidades e do NINA-B306. O trabalho segue um fluxo estruturado, é realizada uma revisão da literatura para fundamentar os conceitos centrais relacionados a sistemas embarcados e *firmware*. A partir dessa base teórica, define-se os materiais e o método, bem como o fluxo de desenvolvimento a ser seguido para o desenvolvimento do dispositivo. A seguir, procede-se à implementação do dispositivo e é obtido resultados a partir da abordagem da arquitetura em camadas.

A relevância deste projeto se justifica pelo crescimento das aplicações em sistemas embarcados e Internet das Coisas (IoT), que se tornam cada vez mais sofisticadas e inteligentes (SmartSoC, 2024).

Por fim, este trabalho pretende contribuir para os avanços na criação de sistemas embarcados, explorando as possibilidades de implementação de dispositivos inteligentes, e destacando a importância de uma abordagem que busque entender a fundo o funcionamento dos microcontroladores e módulos utilizados no desenvolvimento.

2 OBJETIVOS

Este capítulo apresenta o objetivo geral e os objetivos específicos deste trabalho.

2.1 OBJETIVO GERAL

O objetivo geral deste trabalho é explorar as capacidades do Open CPU do NINA-B306, desenvolvendo um dispositivo para aquisição de dados que irá registrar a temperatura ambiente, a fim de entender as funcionalidades e aplicabilidades do módulo em sistemas embarcados.

2.2 OBJETIVOS ESPECÍFICOS

Os objetivos específicos a serem alcançados são:

- Compreender as funcionalidades básicas do módulo NINA-B306, analisando as características que possibilitam sua utilização em projetos de sistemas embarcados.
- Analisar e implementar os processos da arquitetura em camadas no desenvolvimento do dispositivo, identificando as interações entre os *drivers* de baixo e alto nível e suas contribuições para o sistema.
- Criar um protótipo funcional do dispositivo para aquisição de dados em uma *proto board*, implementando um código em linguagem C que permita a aquisição e o armazenamento de dados de temperatura, demonstrando na prática as capacidades do módulo NINA-B306.

3 REVISÃO DE LITERATURA

Neste capítulo, será realizada uma apresentação dos conceitos fundamentais relacionados aos temas centrais deste trabalho, os quais contribuirão para a sua elaboração. Serão explorados aspectos como sistemas embarcados, *firmware* e *Bluetooth Low Energy*.

3.1 SISTEMAS EMBARCADOS

Os sistemas embarcados são dispositivos computacionais projetados para realizar tarefas específicas e dedicadas, geralmente sem a interação contínua do usuário. De acordo com Lacamera (2023), essa classe de sistemas utiliza uma arquitetura baseada em microcontroladores, caracterizando-se por recursos limitados que, no entanto, possibilitam a construção de sistemas em tempo real, graças às funcionalidades oferecidas pela própria arquitetura de *hardware*. A diversidade de mercados e tecnologias resulta em uma variedade de formatos e tamanhos para esses dispositivos, mas é comum que apresentem dimensões compactas e uma quantidade restrita de recursos.

Diferentes pessoas podem interpretar o conceito de sistemas embarcados de maneiras variadas. Para um profissional que trabalha com servidores, uma aplicação desenvolvida para um telefone pode ser considerada um sistema embarcado. Em contraste, alguém que programou microcontroladores de 8 bits pode não classificar como embarcados dispositivos que operam com sistemas operacionais (White, 2024). “Um sistema embarcado é um sistema computacional construído especificamente para uma aplicação” (White, 2024, p. 1). Essa definição reflete a missão desses sistemas em comparação a um computador de propósito geral, levando a um menor suporte para funcionalidades que não estão diretamente ligadas à execução da tarefa para a qual foram projetados.

As limitações dos sistemas embarcados não se restringem apenas ao hardware, mas também se estendem ao software. De acordo com White (2024), alguns sistemas precisam operar de maneira determinística, garantindo um comportamento previsível em todas as execuções, ou em tempo real, reagindo rapidamente a eventos. Exemplos dessa necessidade incluem sistemas críticos, como monitores cardíacos, que não podem falhar de maneira silenciosa e devem fornecer

mensagens de erro claras ao primeiro sinal de problemas. Outros sistemas podem precisar ser tolerantes a falhas, oferecendo uma degradação graciosa em face de erros, especialmente em situações em que a manutenção seja inviável, como em satélites ou dispositivos de rastreamento em ambientes remotos.

Assim, a intersecção entre *hardware* e *software* em sistemas embarcados é complexa e exige um profundo entendimento dos requisitos funcionais e das restrições envolvidas Lacamera (2023) e White (2024) ressaltam que a eficiência e a especificidade dessas plataformas são fundamentais para o funcionamento eficaz em diversos contextos, desde eletrodomésticos até aplicações mais sofisticadas em automóveis, consolidando a importância dos sistemas embarcados na atualidade.

3.1.1 Arquitetura

A arquitetura de um sistema embarcado é fundamentalmente centrada em seu microcontrolador, também conhecido como *microcontroller unit* (MCU). Essa centralização em torno do MCU permite que o sistema realize suas funções específicas de maneira eficiente, otimizando o uso dos recursos disponíveis (Lacamera, 2023). Além disso, a escolha da arquitetura impacta diretamente o desempenho do sistema, pois um microcontrolador adequado pode permitir a implementação de estratégias de *design* que melhor atendam às necessidades da aplicação em questão.

Na concepção de sistemas embarcados modernos, a eliminação de práticas desatualizadas é essencial. Twomey (2024) argumenta que, como regra geral, circuitos analógicos e o processamento de sinal devem ser minimizados sempre que possível, dado que esses componentes podem introduzir problemas relacionados a ruído, distorção e integridade do sinal. Embora nunca se possa evitar completamente a interação com o "mundo analógico" — que é acessível através de uma variedade de sensores — a eficiência do processamento desses sinais é crucial para garantir a funcionalidade e a confiabilidade do sistema embarcado.

Esses sistemas são caracterizados por serem específicos em suas tarefas, integrando *hardware*, *software* e *firmware* de maneira coesa. Segundo Gillis e Lutkevich (2024), apesar da diversidade em sua complexidade, sistemas embarcados geralmente se compõem de alguns elementos principais: *hardware*, que é baseado em microprocessadores e microcontroladores; *software* e *firmware*, que pode variar

em complexidade e geralmente requer pouca memória. Essa configuração é importante para a efetividade do sistema, pois permite que cada componente trabalhe para cumprir a função para a qual foi projetado.

3.1.2 Microcontroladores

Os microcontroladores são componentes essenciais em sistemas embarcados, especialmente em soluções de menor escala, onde a funcionalidade do computador é frequentemente incorporada em um único *chip*. Segundo Catsoulis (2005), esses dispositivos são caracterizados pela inclusão de uma unidade central de processamento (CPU), uma quantidade reduzida de memória interna (ROM e RAM) e uma interface de entrada/saída (I/O), que é implementada como blocos de subsistema dentro do microcontrolador. Esses subsistemas fornecem funcionalidades adicionais para o processador e são comuns em diversos modelos, permitindo uma integração eficiente e compacta.

Sendo assim, Lacamera (2023) enfatiza que o MCU é tipicamente um circuito integrado que contém não apenas o processador, mas também memória RAM, memória *flash*, receptores e transmissores seriais, além de outros componentes essenciais. Com diferentes arquiteturas disponíveis no mercado, os microcontroladores são projetados para serem econômicos, de baixo consumo energético e com diversas funcionalidades integradas, sendo muitas vezes referidos como Sistemas em *Chip* (SoC). Essa flexibilidade permite que alguns microcontroladores sejam dedicados a aplicações específicas, expondo interfaces particulares para comunicação com periféricos e o mundo exterior, enquanto outros são otimizados para custos reduzidos e menor consumo de energia. A Figura 1 exemplifica a ideia do interior de um microcontrolador em um diagrama de blocos.

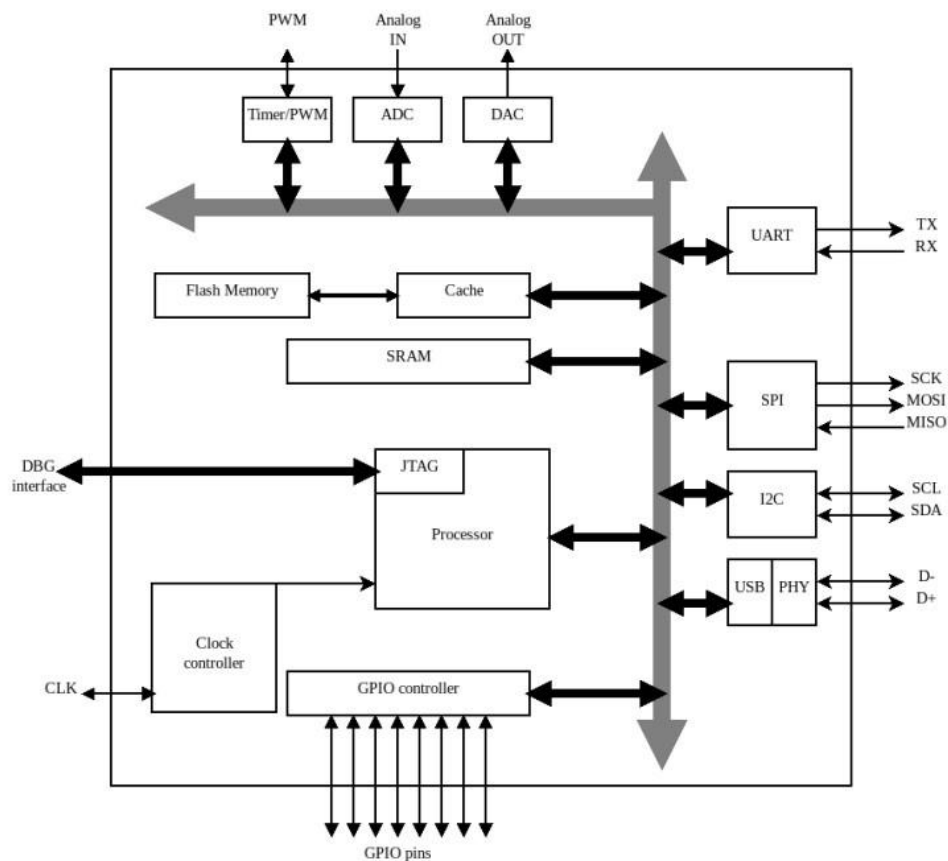


Figura 1: Um diagrama de blocos simplificado dos componentes dentro de um microcontrolador genérico

Fonte: Lacamera, (2023)

Além disso, a escolha de um microcontrolador adequado é um fator determinante para o sucesso de um projeto de sistema embarcado. Lacamera (2023) ressalta que a capacidade de processamento, as opções de memória e as interfaces de comunicação disponíveis influenciam diretamente a funcionalidade e a eficiência da aplicação. Assim, um projeto bem-sucedido deve considerar as características específicas do microcontrolador em relação às exigências do sistema, garantindo que todos os componentes trabalhem em conjunto de maneira harmoniosa. Dessa forma, os microcontroladores não apenas desempenham um papel funcional, mas também constituem a espinha dorsal da arquitetura dos sistemas embarcados modernos, permitindo uma vasta gama de aplicações em diferentes setores.

3.1.3 Periféricos e interfaces de comunicação

Os microcontroladores modernos apresentam uma variedade de recursos que são fundamentais para a construção de sistemas embarcados estáveis e confiáveis.

Lacamera (2023) observa que, após a inicialização do sistema, tanto a memória quanto os periféricos podem ser acessados, permitindo a ativação dos componentes associados por meio de registradores específicos do sistema. Essa configuração envolve a definição das frequências corretas para as linhas de *clock* e a ativação de interrupções, elementos para a comunicabilidade e funcionalidade integradas do sistema. Com essa abordagem, os microcontroladores tornam-se capazes de interagir com dispositivos externos de maneira eficiente, estabelecendo uma comunicação robusta.

Para facilitar a comunicação com periféricos e outros microcontroladores, existem padrões bem estabelecidos no mundo dos sistemas embarcados. Segundo Lacamera (2023), muitos dos pinos externos dos microcontroladores podem ser programados para executar comunicações utilizando protocolos específicos, como a comunicação assíncrona UART, o barramento *Serial Peripheral Interface* (SPI) e o barramento *Inter-Integrated Circuit* (I2C). Esses protocolos são amplamente utilizados devido à sua simplicidade e eficiência, permitindo que os microcontroladores se conectem a outros dispositivos, como sensores, memórias externas e módulos de comunicação.

Além da comunicação padrão, a capacidade de utilizar entradas e saídas digitais (GPIO) também é uma característica dos microcontroladores. Catsoulis (2005) destaca que essas portas podem ser configuradas através de *software*, proporcionando flexibilidade para funcionarem como entradas digitais para ler estados de botões ou como saídas digitais para acionar dispositivos externos, como motores ou luzes. Essa versatilidade possibilita a síntese de interfaces e protocolos personalizados, adequando o microcontrolador às necessidades específicas da aplicação. A combinação dessas funcionalidades com interfaces seriais especializadas como SPI e I2C aumenta significativamente as oportunidades de integração e comunicação em sistemas embarcados, tornando-os adaptáveis a diversas situações e requisitos.

3.1.4 Open CPU

O conceito de *Open CPU* refere-se a uma solução de desenvolvimento embutido, onde módulos podem ser utilizados como o processador principal. Conforme descrito no guia da Quectel (2013), essa tecnologia foi projetada para

facilitar o *design* e acelerar o desenvolvimento de aplicações, permitindo que soluções inovadoras sejam incorporadas diretamente em alguns módulos, eliminando a necessidade de um microcontrolador externo. Essa flexibilidade é amplamente explorada em uma variedade de setores, incluindo rastreamento, automotivo e energético, ilustrando a versatilidade do *Open CPU* em múltiplas aplicações.

A Figura 2 ilustra um exemplo da arquitetura de *software* do *Open CPU*, destacando interfaces de multiplexação. Embora o exemplo da Quectel represente uma aplicação específica do conceito de *Open CPU* para seu próprio produto, essa abordagem é aplicável a diversos módulos e outras soluções no campo, ampliando consideravelmente as possibilidades de desenvolvimento em sistemas embarcados Quectel (2013). Segundo a Fibocom (2022), o *Open CPU* não apenas reduz o consumo de energia dos dispositivos com seu tamanho compacto, mas também simplifica o *design* de circuitos ao reduzir a quantidade de dispositivos periféricos, facilitando o trabalho dos desenvolvedores e melhorando o custo-benefício dos dispositivos finais.

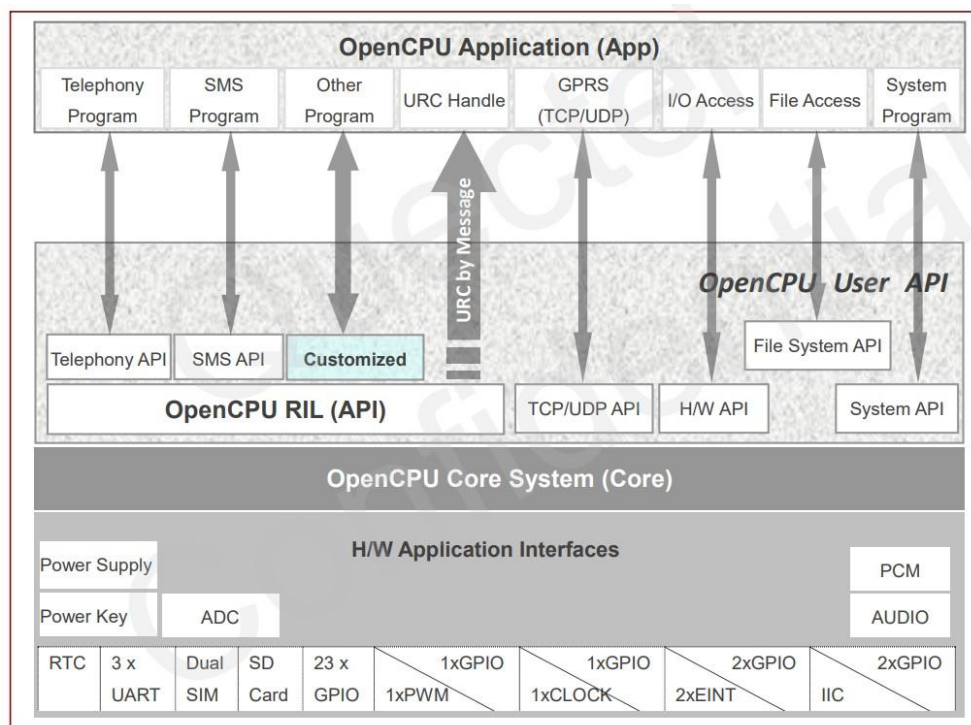


Figura 2: Exemplo da arquitetura de *software* *Open CPU*

Fonte: Quectel (2013).

3.2 FIRMWARE

O *firmware* desempenha um papel relevante na operação de dispositivos de *hardware*, funcionando como um *software* que garante a execução de tarefas, como entrada e saída de dados, além de fornecer as instruções necessárias para a comunicação entre os dispositivos. Segundo Kovacs (2021), esse tipo de *software* é geralmente projetado para operar em um nível elevado e é considerado “parcialmente permanente”, uma vez que suas funcionalidades permanecem inalteradas até que uma atualização seja realizada pelo usuário. Essa característica destaca a importância do *firmware* na funcionalidade de um dispositivo, evidenciando que sua manutenção pode ser necessária para a continuidade das suas operações.

No contexto da engenharia de *firmware*, Davidson e Shriver (1978) afirmam que, de maneira análoga à engenharia de *software*, esta disciplina abrange a especificação e o *design* de micro programas, além das técnicas de construção, depuração, teste, verificação e manutenção. O primeiro passo em qualquer projeto de *software* ou *firmware* é a definição dos objetivos a serem alcançados, que requer uma interação direta com os usuários finais do sistema. Essa etapa é fundamental para obter uma compreensão clara de fatores como custo aceitável, interfaces com outros sistemas e o tamanho do sistema a ser desenvolvido.

3.2.1 Drivers e código reutilizável

Os *drivers* desempenham uma função essencial na arquitetura de sistemas embarcados, sendo responsáveis pela inicialização e gerenciamento do *hardware*. De acordo com Noergaard (2013), esses *softwares*, conhecidos como *drivers* de dispositivo, atuam como intermediários entre o *hardware* e as camadas superiores de *software*, incluindo o sistema operacional e aplicações. Cada sistema embarcado que exige *software* possui, no mínimo, essa camada básica de *drivers*, que garantem que o *hardware* seja corretamente acessado e controlado.

A reutilização de código em sistemas embarcados é uma prática que deve ser cuidadosamente considerada durante o desenvolvimento. Ayinde (2022) enfatiza que o *design* e a arquitetura incorporados em um *firmware* devem permitir a portabilidade e reutilização do código entre diferentes plataformas de *hardware*. A adoção de uma arquitetura de *software* em camadas, que inclua camadas para aplicação, *software* e

hardware é crucial para facilitar essa reutilização. No contexto dessa arquitetura, os engenheiros de software embarcado podem criar um código mais adaptável, onde a camada de aplicação depende da camada de *drivers*, permitindo que os programadores e usuários finais interajam com o sistema sem a necessidade de conhecer detalhes técnicos dos microcontroladores. A Figura 3, demonstra essa ideia.

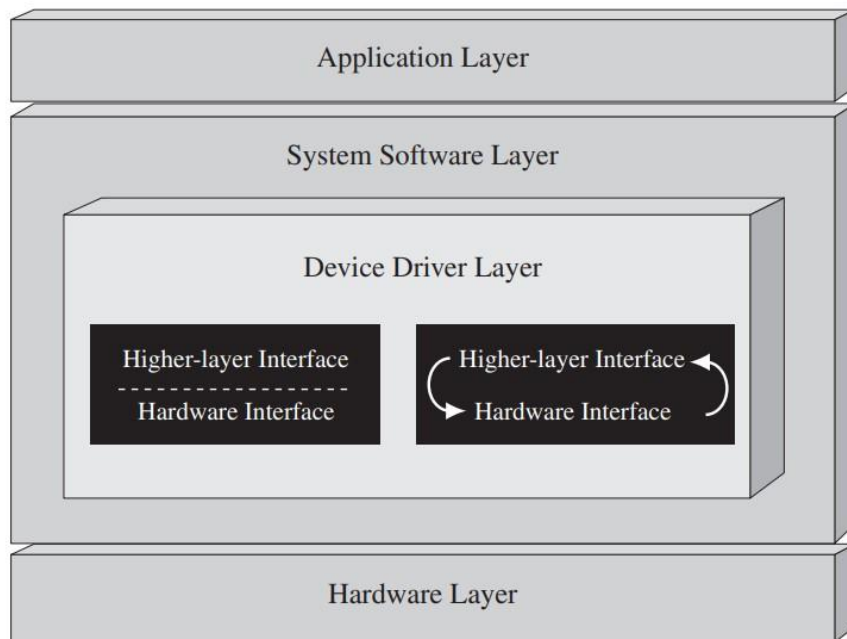


Figura 3: Camadas de código

Fonte: Noergaard (2013)

Além disso, Noergaard (2013) ressalta que os *drivers* de dispositivo podem ser classificados em dois tipos principais: específicos para a arquitetura do sistema ou genéricos. Os *drivers* específicos gerenciam hardware integrado ao processador mestre, enquanto os genéricos atuam em *hardware* que não está incorporado diretamente no processador. Essa distinção é fundamental para o desenvolvimento de sistemas embarcados, pois diferentes tipos de *hardware* requerem diferentes níveis de suporte em seus *drivers*, influenciando diretamente a forma como o *software* se comunica e interage com o *hardware*.

A evolução da programação em sistemas embarcados também reflete a necessidade de um *design* mais estruturado. Beningo (2017) aponta que, em um passado não muito remoto, a falta de melhores práticas e arquiteturas de *software* resultou em códigos desorganizados, frequentemente comparados a um “código espaguete”. Essa abordagem, caracterizada por uma mescla caótica de funções,

impediu a reutilização efetiva e a manutenção do *firmware*. Em contrapartida, a ênfase em uma arquitetura em camadas permite que os desenvolvedores criem códigos claros e estruturados, como uma lasanha, que é facilmente compreensível e passível de alterações nas diferentes camadas sem comprometer a integridade do sistema global.

3.2.2 *Bootloader*

O *bootloader* é um componente muitas vezes presente no processo de inicialização de sistemas embarcados, exigindo dos engenheiros um conhecimento aprofundado sobre o funcionamento do processador e seu gerenciamento de memória. Beningo (2015) destaca que o desenvolvimento de um *bootloader* pode ser desafiador, mas que, uma vez compreendido, se torna progressivamente mais fácil implementar novos *bootloaders* devido à adoção de abordagens consistentes de *design*. Esse *software* especializado opera em um espaço de memória compartilhado com outras aplicações, desempenhando funções que vão desde a inicialização do processador até a programação de novas aplicações em um sistema.

A flexibilidade e funcionalidade dos *bootloaders* são amplamente reconhecidas, especialmente no que diz respeito à atualização de software em dispositivos instalados em locais remotos. Lacamera (2023) enfatiza que essa capacidade de atualizar *firmware* após a implantação oferece vantagens significativas em termos de manutenção, permitindo a correção de bugs e a introdução de novas funcionalidades. Além disso, os *bootloaders* podem implementar mecanismos de atualização remota automática, verificação da integridade da imagem do aplicativo e até mesmo mecanismos que lidam com falhas.

Existem diferentes modelos de comportamento para *bootloaders*, que influenciam diretamente como são projetados e implementados nas aplicações. Beningo (2015) descreve dois modelos principais: um modelo automatizado, em que o *bootloader* gerencia seu próprio processo de atualização, e um modelo que depende de instruções externas, geralmente controladas por um aplicativo de *software* em um computador. No segundo modelo, o *bootloader* aguarda comandos externos para iniciar o processo de *flash* de novos *firmwares*, refletindo a necessidade de um design que atenda não apenas à funcionalidade, mas também às limitações de espaço no sistema.

3.3 BLUETOOTH LOW ENERGY

O *Bluetooth Low Energy* (BLE) se destaca como um padrão de comunicação sem fio projetado para operar com baixa demanda de energia, sendo especialmente útil em dispositivos vestíveis e equipamentos médicos. Desde seu lançamento pela *Bluetooth SIG* em 2010, em conjunto com o *Bluetooth* 4.0, o BLE proporciona uma autonomia significativamente maior em comparação ao *Bluetooth* clássico. Embora as versões anteriores do *Bluetooth* já apresentassem características de baixo consumo, o BLE foi desenvolvido especificamente para permitir que dispositivos com baterias reduzidas funcionem por longos períodos sem a necessidade de recarga, o que é essencial em aplicações onde o acesso a uma fonte de energia é limitado (Alecrim; Marques, 2023).

Além da eficiência energética, o *Bluetooth Low Energy* também se distingue por seu potencial de integração em uma ampla gama de dispositivos já existentes. De acordo com Gupta (2016), a prática adoção dessa tecnologia em dispositivos móveis, como celulares, que têm uma taxa de incorporação de quase 100%, permite que esses dispositivos atuem como ponte para sensores compatíveis com BLE, promovendo uma diversidade de aplicações a um custo incremental baixo. Essa característica torna o BLE não apenas um facilitador de conectividade, mas também um catalisador para inovações no mercado de sensores sem fio.

É importante destacar que o *design* do *Bluetooth Low Energy* é fundamentado na combinação de baixo custo e consumo eficiente. Desde sua concepção, o BLE foi otimizado para atender às demandas de dispositivos que operam com baterias, permitindo uma produção em larga escala a um custo acessível. Essa abordagem contrasta com outras tecnologias que priorizam altas taxas de transferência de dados, mostrando que o BLE é mais adequado para aplicações que não requerem um fluxo contínuo de informações, mas sim transferências de dados em curtos períodos (Heydon, 2013).

3.3.1 Arquitetura

A arquitetura do *Bluetooth Low Energy* (BLE) é fundamentada em uma estrutura de camadas que organiza distintos níveis de comunicação e funcionalidade. A arquitetura é composta por três componentes principais: controlador (*controller*),

host e aplicativos (*Apps*). O controlador é um dispositivo físico responsável pela transmissão e recepção de sinais de rádio, enquanto o *host* é uma pilha de *software* que gerencia a comunicação entre dispositivos e a oferta de vários serviços simultaneamente, os aplicativos utilizam essa pilha de *software* para habilitar casos de uso específicos (Heydon, 2013). A simplificação na arquitetura do BLE em comparação ao *Bluetooth* clássico reflete um *design* voltado para a eficiência energética, que é uma das características mais notáveis desta tecnologia. A Figura 4, exemplifica essa arquitetura de camadas.

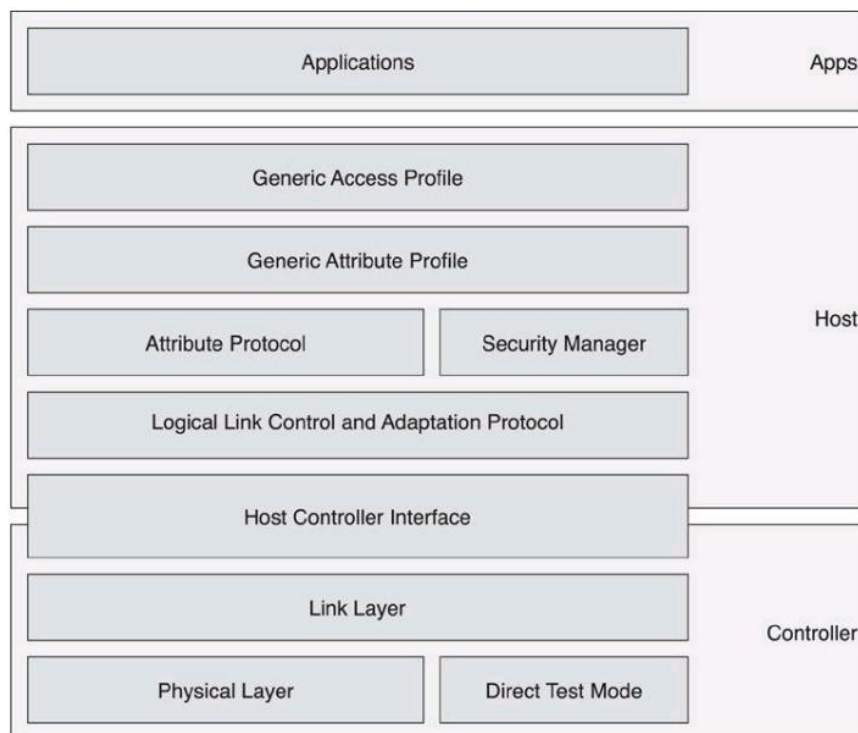


Figura 4: A arquitetura do BLE

Fonte: Heydon (2013)

Dentro da camada do controlador, incluem-se o *Physical Layer* (PHY) e o *Link Layer*, além de um modo de teste direto e a interface de controle do *host* (*Host Controller Interface* - HCI) (Afaneh, 2022).

De acordo com Heydon (2013), o PHY é responsável pela modulação e demodulação dos sinais que operam na faixa de 2,4 GHz, utilizando os métodos para codificar os bits. O *Link Layer*, por sua vez, gerencia a criação e manutenção de conexões, além de garantir que os pacotes sejam estruturados corretamente e que as sequências de verificação e de criptografia estejam em conformidade com as especificações do BLE (Gupta, 2016).

Uma característica importante da arquitetura do BLE é o uso de canais publicitários e canais de dados. O BLE opera com apenas três canais dedicados para publicidade, em contraste com os 32 canais disponíveis no *Bluetooth* clássico, o que resulta em conexões mais rápidas e menor consumo de energia. Quando um dispositivo BLE deseja ser descoberto, ele transmite sinais nesses canais, economizando tempo e energia ao evitar a varredura em múltiplos canais (Hlapisi, 2023). Essa abordagem é importante para aplicações que requerem a troca esporádica de pequenas quantidades de dados, como monitores de frequência cardíaca ou sensores de temperatura.

Segundo Bin Aftab (2017), além da eficiência de energia, a arquitetura do BLE propõe uma abordagem modular, onde diferentes camadas são responsáveis por funções específicas, permitindo um *design* mais eficiente e a implementação de recursos avançados de segurança e gerenciamento. Essa arquitetura não só otimiza a comunicação em cenários de baixo consumo, mas também assegura que uma ampla gama de dispositivos possa se comunicar dentro de um ecossistema sem fio e em expansão.

3.3.2 Modelo de dados e GATT

O modelo de dados do *Bluetooth Low Energy* é estruturado em torno do *Generic Attribute Profile* (GATT), que define como os dados são organizados e acessados. O GATT é essencial para a comunicação entre dispositivos e é fundamentado no *Attribute Protocol* (ATT), que estabelece as regras para leitura e escrita de atributos em um dispositivo remoto. Cada atributo é identificado por um endereço único conhecido como UUID, e o GATT permite a criação de perfis específicos que agrupam características relacionadas a serviços (Bin Aftab, 2017). Dessa forma, o GATT facilita um modelo de comunicação consistente que assegura que todos os dispositivos podem interagir de maneira previsível.

Um perfil GATT é composto por serviços e características, onde os serviços agrupam um ou mais atributos que atendem a uma funcionalidade específica. Por exemplo, um serviço de monitoramento de frequência cardíaca pode incluir características como Medição da Frequência Cardíaca e Localização do Sensor (Hlapisi, 2023).

Segundo Heydon (2013), as características são a unidade básica de dados que pode ser lida ou escrita, permitindo que um dispositivo cliente interaja efetivamente com os dados expostos por um dispositivo servidor. Essa organização hierárquica dos dados proporciona uma forma eficiente de comunicação entre os dispositivos.

Cada característica não só armazena um valor, mas também inclui informações adicionais através de descritores (*descriptors*). Os descritores são elementos opcionais que oferecem detalhes sobre a característica ou permitem configurar o valor armazenado, por exemplo, um descritor pode definir se uma característica deve gerar uma notificação quando uma medida, como a temperatura, estiver pronta (Gupta, 2016). Essas adições fornecem um nível extra de funcionalidade e ajuste que pode fazer a diferença em aplicações específicas. Para uma melhor visualização, a Figura 5 apresenta a divisão da hierarquia baseada no GATT.

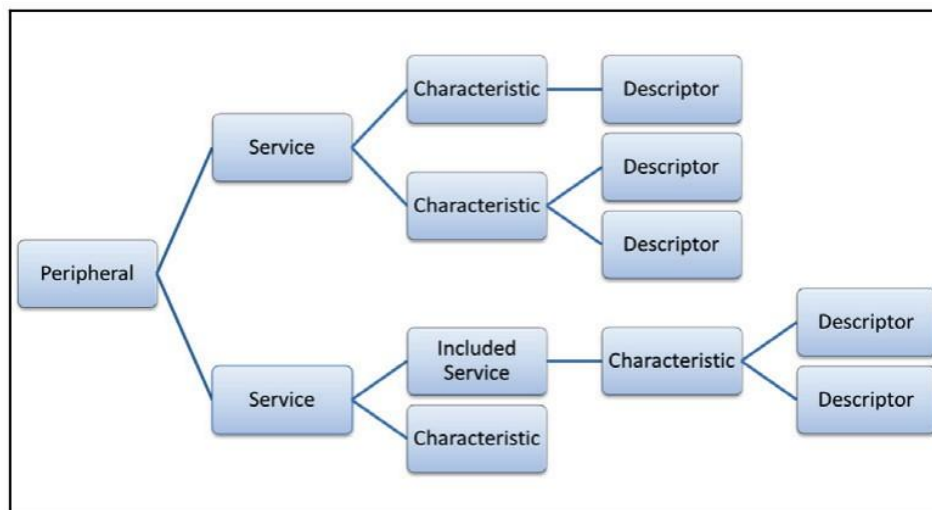


Figura 5: Modelo baseado em GATT

Fonte: Bin Aftab (2017)

O acesso aos atributos é realizado por meio de operações definidas pelo GATT, que incluem comandos, solicitações, respostas, notificações e indicações. Essas operações possibilitam tanto a leitura quanto a escrita de atributos, garantindo que os dispositivos possam comunicar informações de maneira eficaz e responsiva (Gupta, 2016). A simplicidade e a flexibilidade desse modelo de dados são essenciais para a operação em ambientes de baixa energia, onde a economia de bateria é uma prioridade.

De acordo com Gupta (2016), o GATT é projetado para suportar uma dinâmica de interação clara, definida por dois papéis principais: cliente e servidor. O cliente é responsável por iniciar transações com o servidor, enviando comandos e solicitações, enquanto o servidor processa essas requisições e responde com as informações solicitadas. Um dispositivo pode desempenhar ambos os papéis simultaneamente, tornando a estrutura GATT versátil para diversas aplicações.

Além disso, o GATT é projetado para suportar a simplicidade em sua implementação, permitindo que os fabricantes desenvolvam dispositivos de BLE com menor custo e complexidade (Afaneh, 2022). A integração do GATT com o modelo de dados do BLE resulta em uma arquitetura que é tanto robusta quanto adaptável, contribuindo para a ampla adoção dessa tecnologia em uma variedade de aplicações, desde dispositivos vestíveis até sensores de saúde conectados. Com essa estrutura, o *Bluetooth Low Energy* se posiciona como uma solução eficiente e prática para as necessidades de conectividade modernas.

4 MATERIAIS E MÉTODOS

Este capítulo descreve os materiais e a metodologia adotada na investigação das capacidades do *Open CPU* do módulo NINA-B306. Para isso, foi desenvolvido um dispositivo para aquisição de dados que registra a temperatura ambiente em um local específico, permitindo a extração externa das informações para análises posteriores.

4.1 MATERIAIS UTILIZADOS

Os materiais utilizados na realização do projeto incluem o módulo NINA-B306, que será integrado à placa Arduino Nano 33 BLE Sense Revisão 2. Inicialmente, serão utilizados um sensor de temperatura embutido na placa do Arduino, um módulo de memória flash NOR e um conversor serial para USB. Adicionalmente, um gravador J-Link e diversos componentes eletrônicos também serão empregados.

4.1.1 Módulo NINA-B306

O módulo NINA-B306 opera com Bluetooth de baixa energia (BLE) e incorpora um microcontrolador interno. Com dimensões compactas, possui 38 pinos de propósito geral programáveis, oferecendo flexibilidade para diversas aplicações. Suporta interfaces como UART, SPI, I2C (TWI) e conversores analógicos-digital. O módulo conta ainda com um cristal interno de baixa potência de 64 MHz (U-blox, 2023). A Figura 6 apresenta o módulo.



Figura 6: Módulo NINA-B306

Fonte: U-blox (2023)

No núcleo do NINA-B306 está o microcontrolador nRF52840, baseado na arquitetura ARM Cortex-M4, que inclui uma unidade de ponto flutuante (FPU). Este

microcontrolador possui 1 MB de memória *flash* e 256 kB de RAM, o que possibilita o armazenamento e a execução de aplicações complexas. O nRF52840 suporta a interface Bluetooth 5 e possui avançados recursos de segurança, incluindo criptografia AES e um gerador de números aleatórios (Nordic Semiconductor, 2024). O diagrama de blocos do microcontrolador é mostrado na Figura 7.

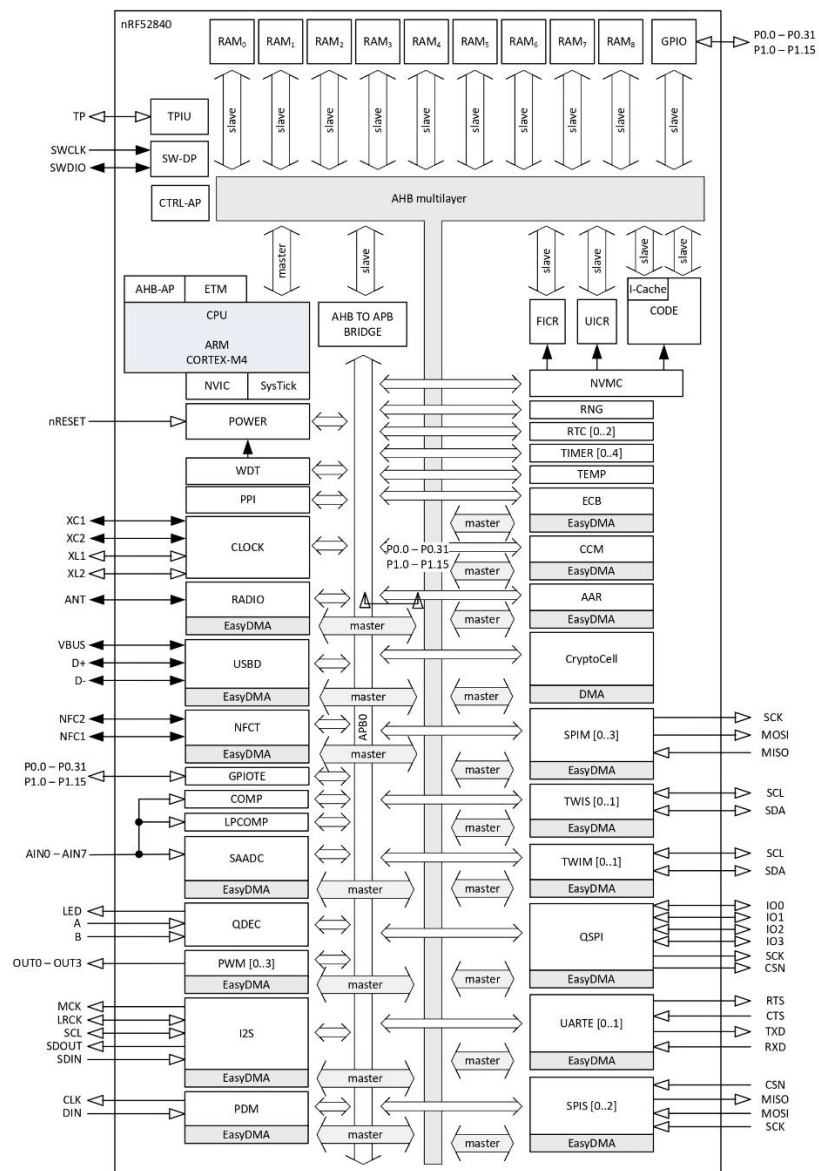


Figura 7: Diagrama de blocos nRF52840

Fonte: Nordic Semiconductor (2024)

4.1.2 Arduino Nano 33 BLE

A placa Arduino Nano 33 BLE Sense Revisão 2 será utilizada para facilitar o acesso ao módulo NINA-B306, simplificando a implementação do sistema inicial. Com

dimensões de 45x18 mm, a placa contém um regulador de alimentação de 3,3V integrado, além de sensores embutidos, incluindo um sensor de temperatura, que será utilizado no dispositivo (Arduino, 2024).

A Figura 8 apresenta a placa. Essa escolha foi feita visando facilitar protótipos e testes iniciais, pois a placa permite o acesso rápido ao módulo NINA-B306 através de conectores facilmente adaptáveis em *protoboards*.

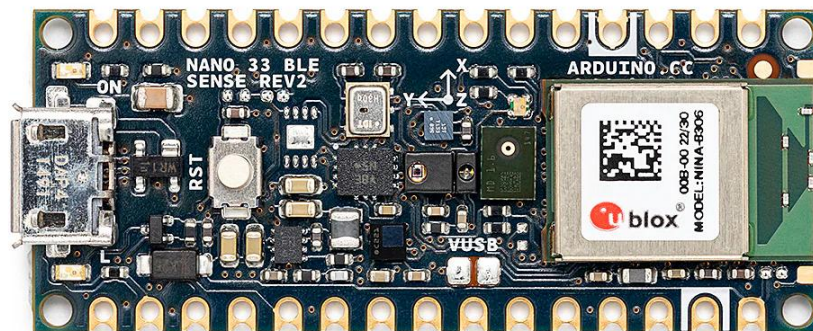


Figura 8: Placa Arduino Nano 33 BLE Sense Revisão 2

Fonte: Arduino (2024)

O sensor de temperatura integrado, HS3003, é um circuito integrado de alta precisão que combina medições de temperatura e umidade relativa em um único dispositivo. Ele é estável e pode medir temperaturas de 0 a 70 °C sem a necessidade de calibração pelo usuário. Possui saída digital via I2C e uma resolução de até 14 bits (Renesas, 2021).

4.1.3 Memória *flash* externa

Uma memória *flash* do tipo NOR, a W25Q64, será utilizada para o armazenamento de dados não voláteis do dispositivo de aquisição. Esta memória possui capacidade de 64 Mbits, com mais de 100 mil ciclos de gravação e pelo menos 20 anos de retenção, usando interface SPI para armazenar e extrair dados (Windbond, 2017).

4.1.4 Conversor serial para USB

Um módulo FTDI com o *chip* FT232RL será utilizado para converter dados USB para serial, permitindo conectar dispositivos com interface TTL ao computador e realizar trocas de dados (FTDI CHIP, 2021).

4.1.5 Outros materiais

Um gravador J-Link será utilizado para programar o microcontrolador via protocolo SWD, possibilitando a programação do *firmware* e a depuração do projeto. Adicionalmente, serão utilizados diversos componentes eletrônicos, como resistores, capacitores, transistores e circuitos integrados, essenciais para a montagem do dispositivo. A seleção desses materiais garantirá a funcionalidade e a confiabilidade do sistema, permitindo a integração entre os diferentes elementos do projeto.

4.2 ESTRUTURA DO SISTEMA EMBARCADO

Seguindo a abordagem em camadas proposta por Noergaard (2013), a Figura 9 ilustra como a aplicação será desenvolvida nesta arquitetura de sistema embarcado, composta por três camadas principais: camada de *hardware*, camada de *software* e camada de aplicação.

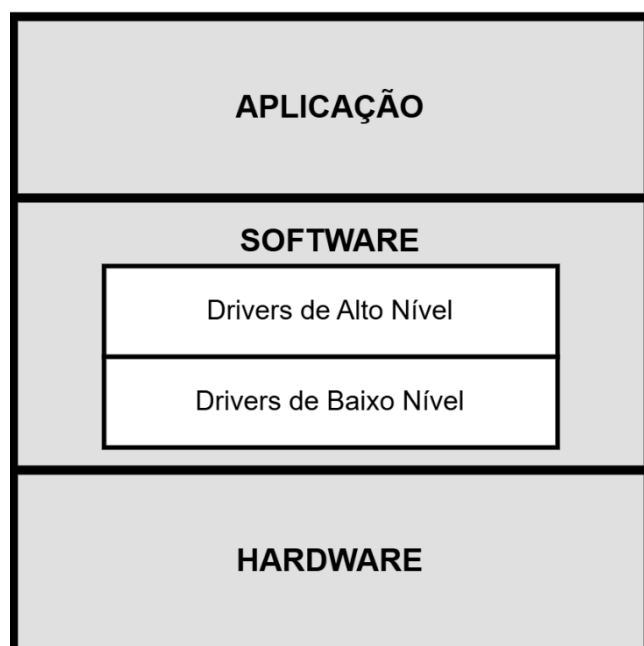


Figura 9: Arquitetura de sistemas embarcados baseada em camadas

Fonte: O autor (2024)

4.2.1 Camada de *hardware*

Nesta camada, serão estabelecidas as conexões elétricas necessárias para o funcionamento do dispositivo. A disposição dos componentes será organizada para garantir eficiência e facilitar a montagem em *protobord*, prevendo uma futura placa de circuito impresso.

4.2.2 Camada de *software*

A camada de *software* é internamente dividida em duas partes:

- *Drivers* de Baixo Nível: Responsáveis pela interação direta com os registradores do microcontrolador, possibilitando o acesso a funcionalidades básicas como controle de pinos e configuração de interfaces de comunicação (UART, SPI e I2C).
- *Drivers* de Alto Nível: Interfaces de *software* que serão acessadas pela camada de aplicação, permitindo uma interação simplificada com o *hardware*.

É importante notar que, ao longo deste trabalho, o termo "*drivers*" é utilizado de forma análoga a "módulos".

4.2.3 Camada de aplicação

A camada de aplicação será composta por um código de alto nível que implementará a inicialização do dispositivo e a lógica de funcionamento, de forma abstrata, permitindo modificações sem a necessidade de um conhecimento detalhado sobre o *hardware* e os *drivers* de baixo nível.

4.3 FLUXO DE DESENVOLVIMENTO

A definição da arquitetura do dispositivo será o primeiro passo, seguida por um processo de desenvolvimento, ilustrado no fluxograma da Figura 10.

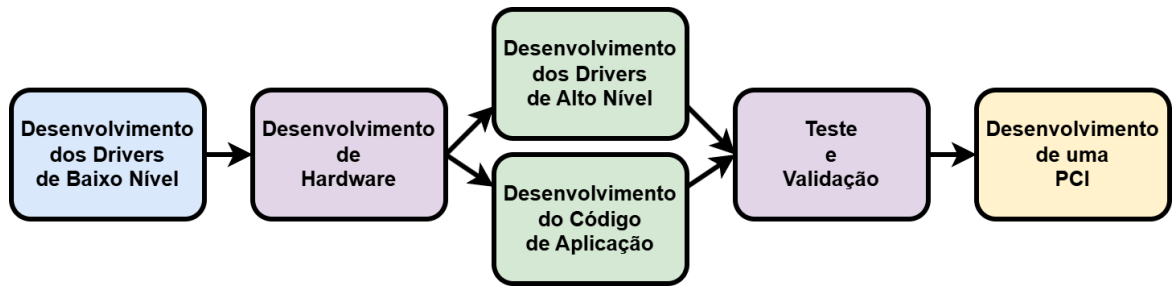


Figura 10: Fluxograma das etapas do desenvolvimento do projeto

Fonte: O autor (2024)

Após a definição da arquitetura em camadas do dispositivo. O fluxo de desenvolvimento iniciará com a criação do repositório da aplicação e a configuração dos arquivos necessários para a compilação do projeto. Nessa fase, os *drivers* de baixo nível serão desenvolvidos para testar as funcionalidades básicas do módulo NINA-B306, etapa essencial para garantir o funcionamento correto do *hardware* desde o início. Depois, o foco se voltará ao desenvolvimento do *hardware*, incluindo a elaboração de um diagrama esquemático simples em forma de blocos e a montagem do circuito em uma *protoboard*, permitindo a validação do projeto até que funcione satisfatoriamente. Todo o código será escrito em C, sem utilizar bibliotecas de abstração para *hardware* de terceiros ou *frameworks* prontos.

Após o desenvolvimento do *hardware*, as etapas de desenvolvimento dos *drivers* de alto nível e do código da aplicação serão conduzidas em paralelo. As funções de *software* serão testadas em conjunto com o *hardware*. A etapa de teste e validação final ocorrerá com o *hardware* montado na protoboard e o *firmware* em execução, permitindo uma avaliação prática do sistema. Após essa validação, a próxima etapa será o desenvolvimento de uma placa de circuito impresso (PCI), apresentando o projeto em um formato de produto real, embora a placa não seja fisicamente produzida, servindo apenas como modelo.

5 RESULTADOS

Os resultados obtidos no desenvolvimento da aplicação serão apresentados a seguir. Primeiro, será descrito a arquitetura do dispositivo, enfatizando os componentes e suas interações. Na sequência, abordará a implementação da arquitetura em camadas, detalhando as especificidades da camada de baixo e de alto nível.

Além disso, serão apresentados testes e validações do dispositivo, com o intuito de garantir sua funcionalidade. Por último, será apresentada a descrição da placa de circuito impresso, que é a concepção física final do dispositivo.

5.1 ARQUITETURA DO DISPOSITIVO

A definição da arquitetura do dispositivo começou com a elaboração de um diagrama de blocos, que pode ser visualizado na Figura 11. Para a aquisição de dados de temperatura, foi utilizado um sensor integrado à placa Arduino escolhida. Adicionalmente, um LED RGB foi incorporado para fornecer uma indicação visual do estado da placa. Externamente, optou-se pelo uso de uma memória *flash* do tipo NOR para a retenção dos dados. Também foi implementado um conversor UART para USB, permitindo a comunicação do módulo NINA com um computador externo, o que viabiliza a troca de informações quando necessário.

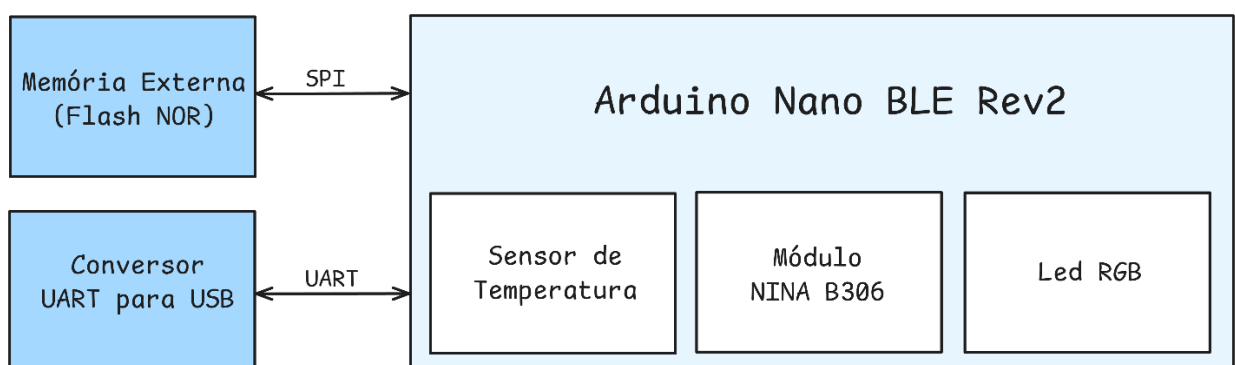


Figura 11: Diagrama de blocos do *hardware* do dispositivo

Fonte: O autor (2025)

A arquitetura de software, apresentada na Figura 12, se baseia na camada de *hardware*, correspondente à parte física do dispositivo, e avança para as camadas de *firmware* do microcontrolador NRF52, que está integrado ao módulo NINA.

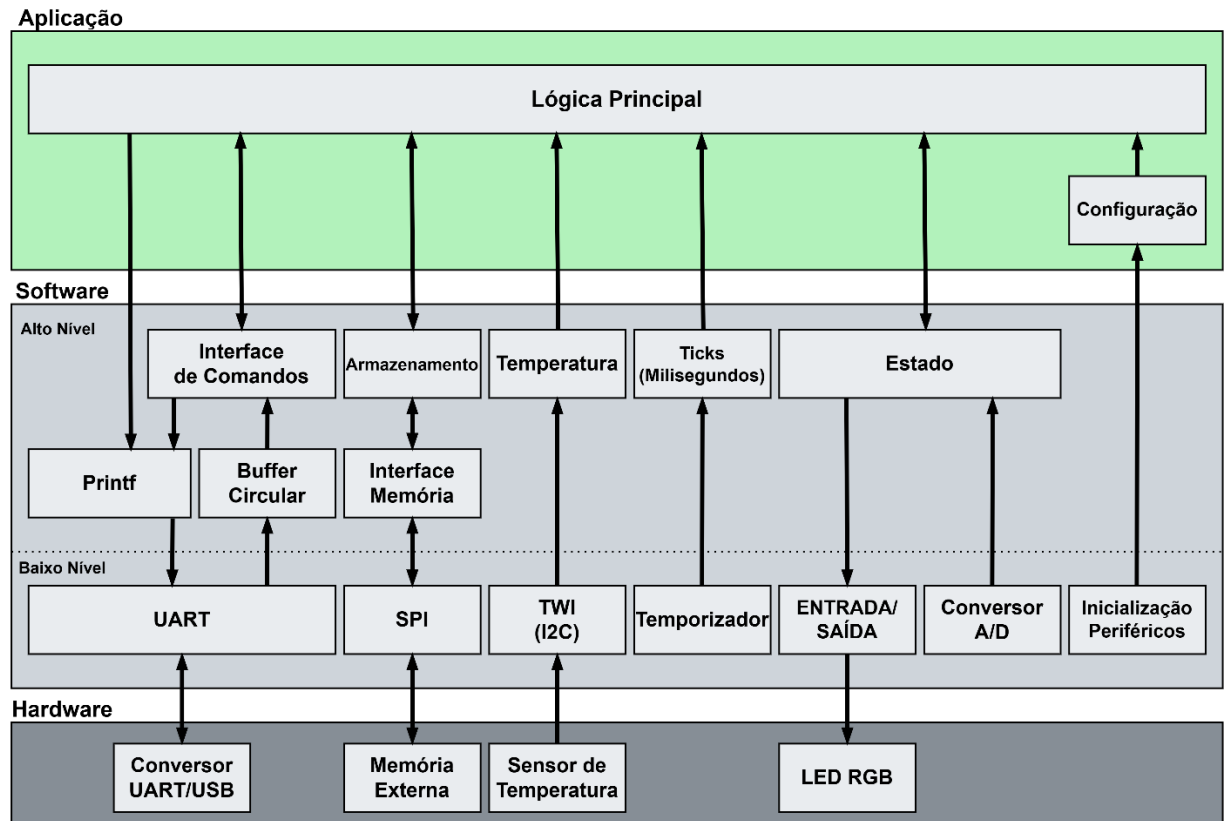


Figura 12: Arquitetura em camadas do dispositivo

Fonte: O autor (2025)

A camada de *software* é dividida em duas partes principais. A primeira é composta por *drivers* de baixo nível, que abrangem todos os periféricos do microcontrolador. Esta abstração visa facilitar a configuração dos principais periféricos necessários, incluindo UART, SPI, TWI, temporizador, entradas e saídas e o conversor analógico para digital. É relevante ressaltar que o periférico TWI também é conhecido como I2C, termos que se referem ao mesmo protocolo. Da mesma forma, o periférico de entradas e saídas é comumente denominado GPIO.

Acima da camada de baixo nível, estabeleceu-se uma camada de alto nível, que contém módulos de código responsáveis por abstrair as principais funcionalidades do dispositivo. Estes módulos incluem a leitura da temperatura, interfaces para armazenamento e extração de informações da memória, e um módulo de estado que verifica se o dispositivo está operando corretamente. Um módulo denominado "*Ticks*" foi desenvolvido para medir o tempo em milissegundos, e também existe uma interface de comandos para facilitar a comunicação com um computador.

Na camada superior, encontra-se a aplicação, que contém a lógica principal do dispositivo em uma estrutura simples e de fácil compreensão, favorecendo a manutenção. Adicionalmente, um módulo de configuração é implementado, sendo está a primeira parte do código, a qual é responsável por iniciar os periféricos de forma adequada.

É importante mencionar que módulos pertencentes à mesma camada podem interagir entre si. Por exemplo, a interface de comandos pode incluir um comando para retornar a temperatura atual que o dispositivo está monitorando.

5.2 IMPLEMENTAÇÃO DA ARQUITETURA EM CAMADAS

Nesta seção, será detalhada a implementação da arquitetura em camadas do dispositivo, que abrange a camada de *hardware*, a camada de baixo nível, a camada de alto nível e a camada de aplicação. No desenvolvimento deste trabalho, todo o código fonte utilizado pode ser acessado na plataforma GitHub. Para visualizar o código completo, o Apêndice D apresenta as instruções necessárias.

5.2.1 Camada de *hardware*

A camada de hardware foi implementada utilizando uma *protoboard*, conforme apresentado na figura 13.

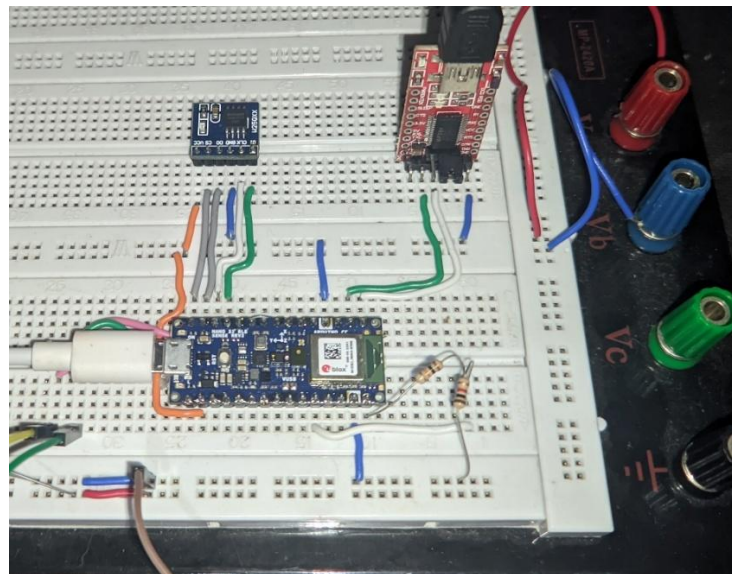


Figura 13: Montagem do dispositivo em *protoboard*

Fonte: O autor (2025)

É importante ressaltar que o diagrama esquemático da montagem em *protoboard* não será incluído, uma vez que uma placa final foi projetada após os testes e validações da montagem. A *protoboard* serviu como uma plataforma de teste inicial, permitindo modificações de conexões de forma rápida e prática.

5.2.2 Camada de baixo nível

A camada de baixo nível foi desenvolvida com o objetivo de estabelecer um padrão de inicialização e operação para os periféricos, facilitando a configuração e utilização. Todo o código foi escrito em linguagem C e, como se trata de um projeto de código aberto, foram utilizados padrões em inglês, visando maior acessibilidade e compreensão por parte de um público amplo.

A Figura 14 apresenta um trecho do código utilizado para a inicialização do periférico GPIO como saída (*output*). Uma estrutura é criada para receber parâmetros pré-definidos no arquivo de cabeçalho "*driver_gpio.h*", e, em seguida, o ponteiro dessa estrutura é passado para a função "*GPIO_Init*", que realiza a inicialização do periférico.

```
GPIO_Handle_t ledPins;
ledPins.pGPIOx = GPIOP0;
ledPins.GPIO_PinConfig.GPIO_PinDir = GPIO_DIR_OUT;
ledPins.GPIO_PinConfig.GPIO_PinNumber = LED_GREEN_PIN;
ledPins.GPIO_PinConfig.GPIO_PinPuPd = GPIO_PIN_NO_PUPD;
ledPins.GPIO_PinConfig.GPIO_InpBuf = GPIO_INP_BUF_DISCONNECT;
ledPins.GPIO_PinConfig.GPIO_PinState = GPIO_PIN_SET;
GPIO_Init(&ledPins);

ledPins.GPIO_PinConfig.GPIO_PinNumber = LED_BLUE_PIN;
GPIO_Init(&ledPins);
```

Figura 14: Inicialização do periférico de GPIO

Fonte: O autor (2025)

Todo o código que implementa a função de inicialização do GPIO é armazenado no arquivo fonte "*driver_gpio.c*", que foi planejado para ser reutilizado com facilidade. A implementação dos periféricos nesta camada requer diversos acessos a registradores do microcontrolador, por isso, o foco foi na abstração e reutilização, de modo que as camadas superiores não precisem lidar diretamente com esses detalhes e que qualquer desenvolvedor que reutilize a camada não precise consultar o *datasheet* do microcontrolador.

A fim de manter a consistência, a Figura 15 apresenta o código utilizado para iniciar o periférico de UART.

```
UART_Handle_t UARHandle;
UARHandle.UART_Config.Baud = UART_STD_BAUD_115200;
UARHandle.UART_Config.Mode = UART_MODE_TXRX;

UARHandle.Tx.port = UART_TX_PORT;
UARHandle.Tx.pin = UART_TX_PIN;
UARHandle.Rx.port = UART_RX_PORT;
UARHandle.Rx.pin = UART_RX_PIN;

UART_Init(&UARHandle);
```

Figura 15: Inicialização do periférico de UART

Fonte: O autor (2025)

Assim como no caso do GPIO, uma estrutura é criada para receber parâmetros pré-definidos, e posteriormente essa estrutura é passada para uma função que realiza a inicialização do periférico.

A Figura 16, por sua vez, ilustra as funções presentes no arquivo de cabeçalho do periférico de GPIO, demonstrando a abordagem de abstrair o acesso aos registradores por meio da criação de funções. Existe a função que inicia o periférico de GPIO e funções que controlam o estado de saída, podendo ser alto ou baixo. Além de, se configurado como entrada, existe a função para ler o estado do periférico.

```
.....

/*
 * Init and De-init
 */
void GPIO_Init(GPIO_Handle_t *pGPIOHandle);
void GPIO_DeInit(GPIOP_RegDef_t *pGPIOx); // not implemented yet

/*
 * Data read and write
 */
uint8_t GPIO_ReadFromInputPin(GPIOP_RegDef_t *pGPIOx, uint8_t PinNumber);
void GPIO_WriteToOutputPin(GPIOP_RegDef_t *pGPIOx, uint8_t PinNumber, uint8_t value);
void GPIO_ToggleOutputPin(GPIOP_RegDef_t *pGPIOx, uint8_t PinNumber);
```

Figura 16: Funções no cabeçalho do periférico de GPIO

Fonte: O autor (2025)

Além dos periféricos de GPIO e UART, também foram desenvolvidos *drivers* de baixo nível para os periféricos de SPI, TWI, conversor analógico-digital e para o *Systick*, que é um temporizador de uso geral.

5.2.3 Camada de alto nível

A camada de alto nível foi implementada para criar uma interface mais intuitiva para acessar os *drivers* de baixo nível, resultando em funções mais abstraídas.

A Figura 17 apresenta o cabeçalho do módulo de temperatura, que inclui a função "*temperature_get*", responsável por retornar o valor de temperatura lido pelo sensor, abstraindo toda a complexidade do protocolo de comunicação e dos comandos necessários para obter essa informação.

```
/*
 * =====
 *      File:      temperature.c
 *      Author:     William Sleman
 *      Description: Temperature header file
 * =====
 */
#ifndef INC_TEMPERATURE_H_
#define INC_TEMPERATURE_H_

#include <stdint.h>

uint16_t temperature_get(void);

#endif /* INC_TEMPERATURE_H_ */
```

Figura 17: Cabeçalho do módulo de temperatura

Fonte: O autor (2025)

Para esclarecer o que foi feito, a Figura 18 mostra a implementação em linguagem C dessa função, onde podemos observar que comandos são enviados e dados são recebidos via TWI, além de cálculos e uma espera de 40 milissegundos para obtenção do resultado. Toda essa complexidade é encapsulada em uma função que pode ser chamada tanto nos módulos da mesma camada quanto diretamente na camada de aplicação.

```

uint16_t temperature_get(void)
{
    uint8_t temperatureAddress = 0x44;
    uint16_t temperature_value = 0;
    float temperature_float = 0;
    uint8_t twi_read[4];
    uint8_t twi_send[1] = {0x00};

    TWI_MasterSendData(TWI1, twi_send, 1, temperatureAddress);

    // 40ms wait time to get measure
    uint64_t start_time = system_get_ticks();
    while((system_get_ticks() - start_time) <= 40);

    TWI_MasterReceiveData(TWI1, twi_read, 4, 0x44);
    temperature_value = (((twi_read[2] << 8) | (twi_read[3])) >> 2 );
    temperature_float = ((temperature_value/16383.0)*165.0 - 40.0)*10;
    temperature_value = (uint16_t)temperature_float;

    return temperature_value;
}

```

Figura 18: Implementação em linguagem C da função "*temperature_get*"

Fonte: O autor (2025)

Seguindo essa lógica, foram desenvolvidos módulos para a interface de comandos, que permitem receber comandos via UART e enviar informações. Também foram criados módulos para armazenamento de dados, junto com a interface da memória, que são utilizados para reter informações no dispositivo. Além disso, o módulo denominado "*Ticks*" foi implementado, o qual fornece um valor que é incrementado a cada milissegundo.

Outro módulo importante é o *buffer* circular, projetado para receber dados da UART de forma segura, evitando a perda de informações. Para garantir o funcionamento adequado do dispositivo, foi criado um módulo de "Estado" que monitora a entrada da alimentação por meio do conversor analógico-digital, identificando possíveis quedas de tensão, o que indicaria um estado de possível falha.

5.2.4 Camada de aplicação

A lógica da camada de aplicação foi elaborada em um código que realiza a aquisição da temperatura a cada 10 minutos, sempre verificando se o estado da placa

está adequado antes de gravar na memória. Essa abordagem previne a corrupção dos dados em caso de falhas na alimentação do dispositivo. A lógica principal do código pode ser visualizada no fluxograma da Figura 19.

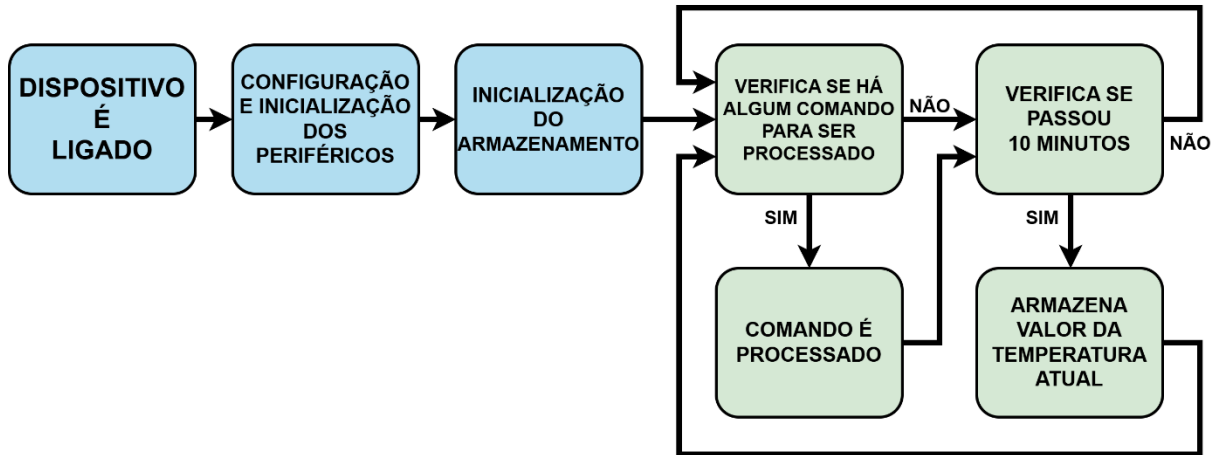


Figura 19: Fluxograma do código principal da camada de aplicação

Fonte: O autor (2025)

É possível extrair os dados armazenados e limpar a memória por meio de comandos via UART. Também é viável, através de um comando, solicitar o retorno da temperatura atual que está sendo lida pelo sensor.

É importante destacar que, além da lógica principal, enquanto a placa estiver em operação, ela sempre estará verificando o seu estado, caso o dispositivo esteja funcionando dentro dos parâmetros operacionais normais, o LED RGB piscará a cada segundo na cor verde. Caso o estado da placa indique uma possível falha, o LED mudará para vermelho, proporcionando uma indicação visual do estado do dispositivo.

O código fonte em C da camada de aplicação, está disponível no Apêndice B.

5.3 TESTES E VALIDAÇÕES

Nesta seção, serão descritos os testes e as validações realizadas no protótipo do dispositivo.

5.3.1 Teste de funcionamento em caso de falha

Para testar a proteção contra a remoção da alimentação, inicialmente a placa foi configurada para operar normalmente. Verificou-se se o LED estava piscando na cor verde, conforme demonstrado na Figura 20.

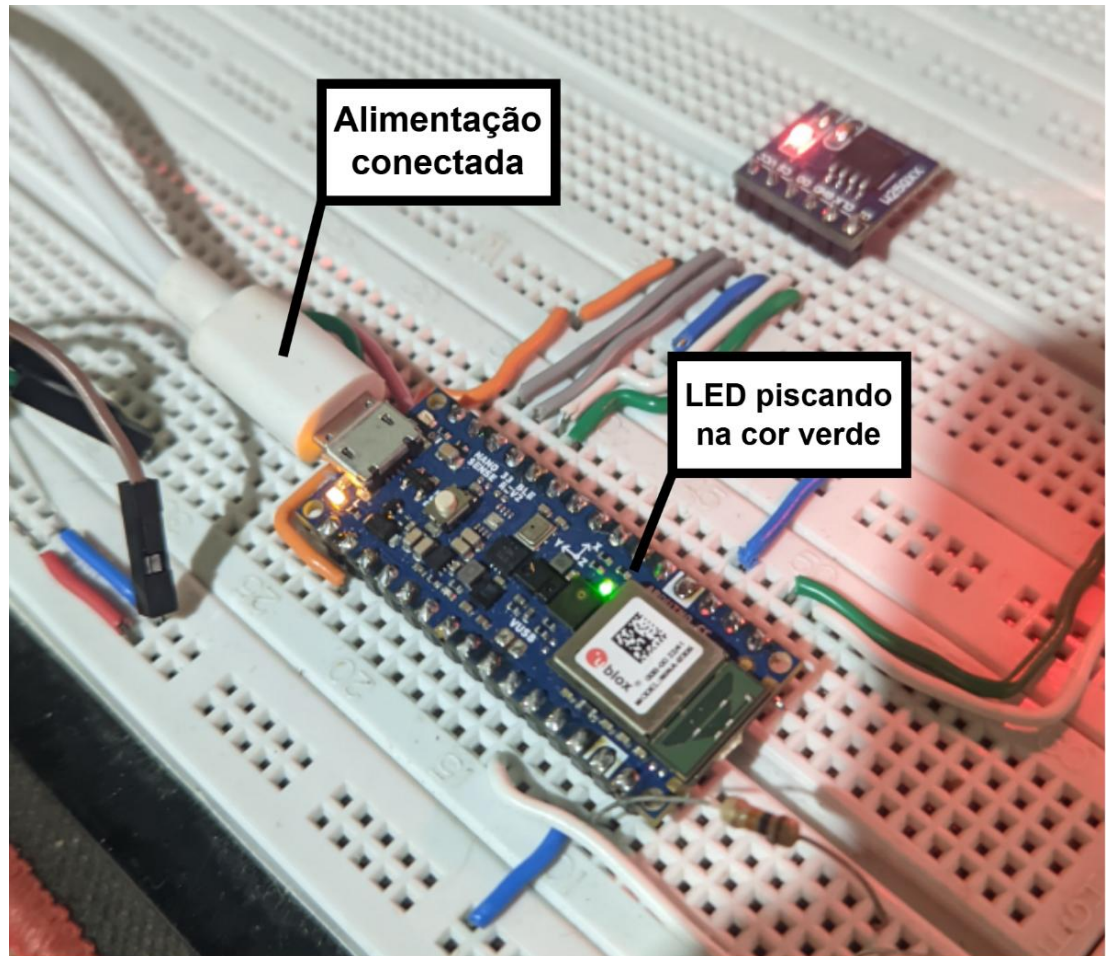


Figura 20: Dispositivo operando normalmente

Fonte: O autor (2025)

Após essa validação, a alimentação foi removida para verificar se o LED indicaria a falha, mudando sua cor para vermelho, como mostrado na Figura 21.

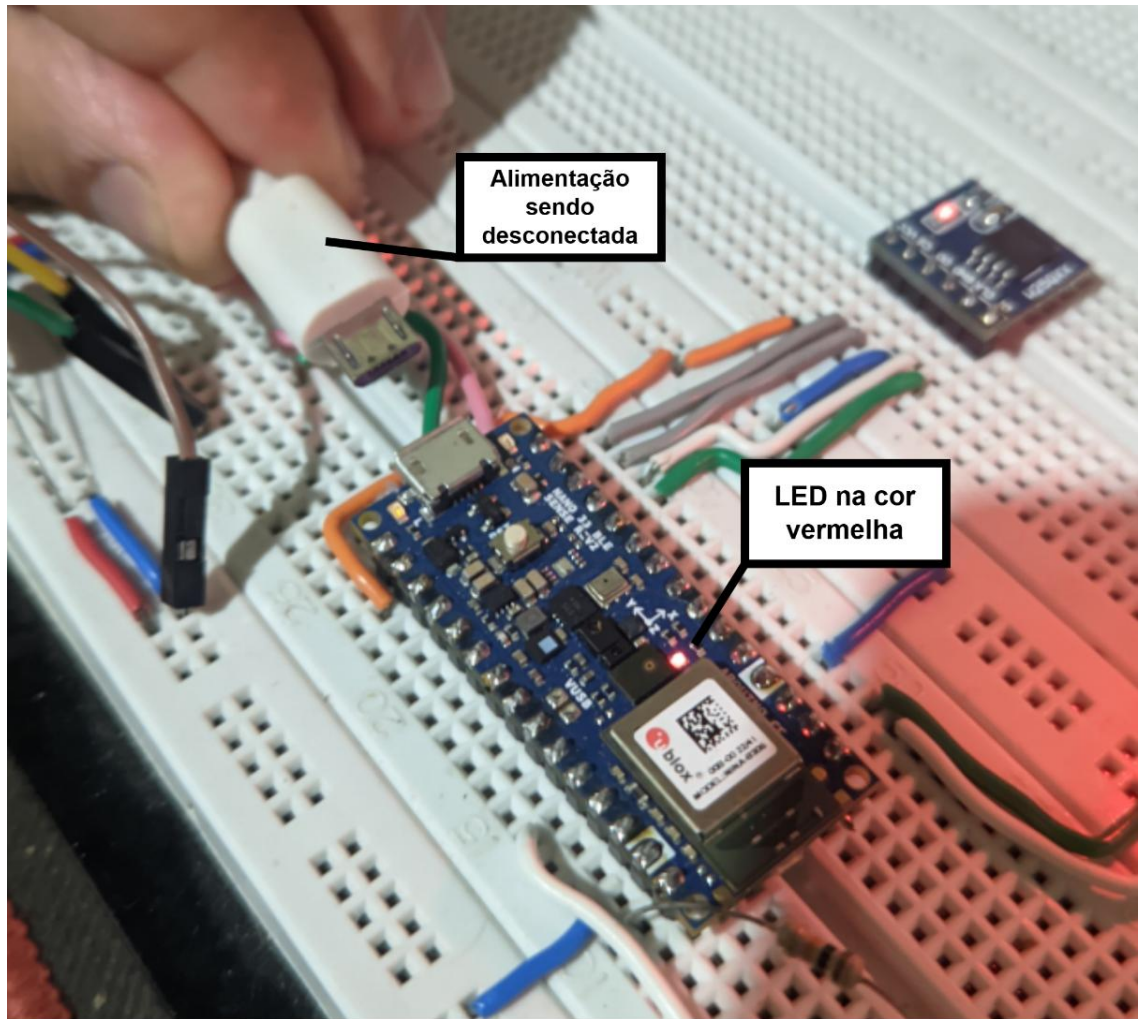


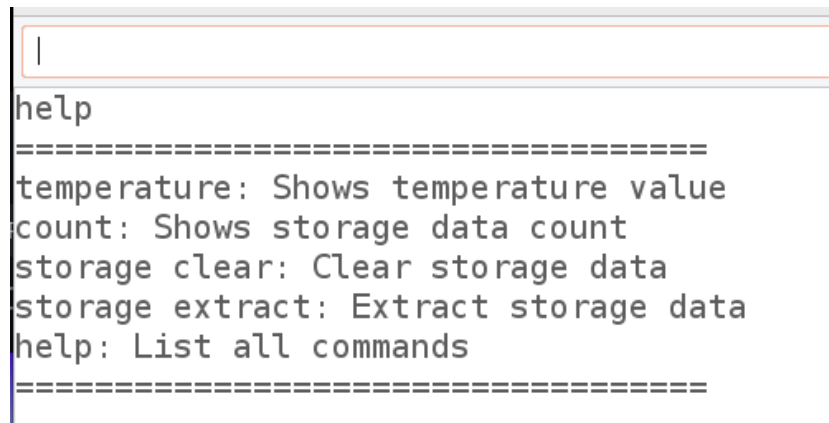
Figura 21: Dispositivo em modo proteção

Fonte: O autor (2025)

Esse teste validou a capacidade do dispositivo de entrar em modo de proteção, evitando que a memória fosse escrita durante a descarga das capacitâncias presentes, prevenindo assim a corrupção dos dados.

5.3.2 Teste dos comandos via UART

Para o teste dos comandos pela UART, o cabo USB foi conectado e utilizada uma interface genérica de comunicação serial. Foi enviado o comando "*help*" para visualizar todos os comandos disponíveis no dispositivo, conforme demonstrado na Figura 22.



```

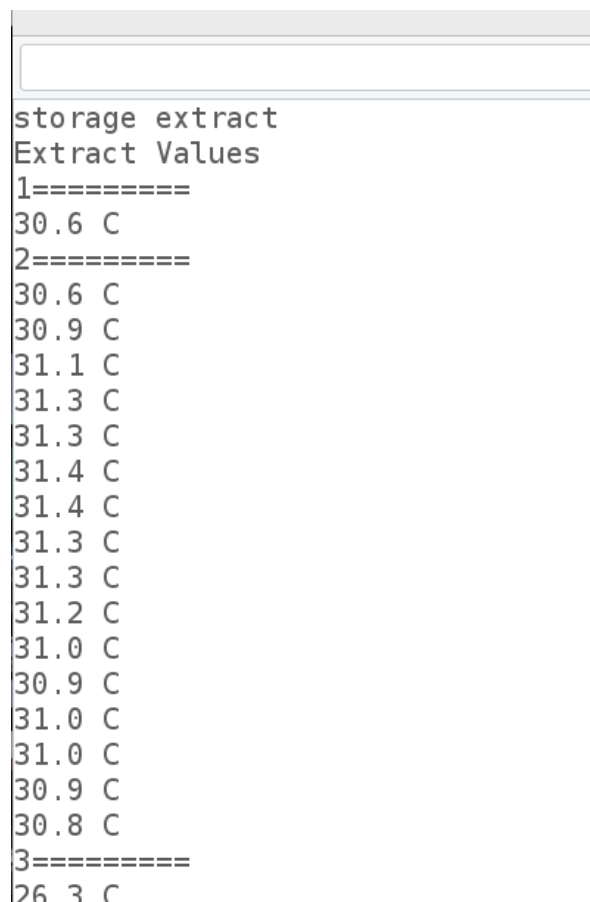
|
help
=====
temperature: Shows temperature value
count: Shows storage data count
storage clear: Clear storage data
storage extract: Extract storage data
help: List all commands
=====

```

Figura 22: Teste comando "*help*"

Fonte: O autor (2025)

Os comandos permitem extrair e limpar os dados coletados pelo dispositivo. Por questões de simplificação, não serão detalhados o funcionamento do processo de gravação na memória nem o armazenamento de dados. A Figura 23 apresenta um teste da extração dos dados, apenas comprovando o funcionamento do comando.



```

storage extract
Extract Values
1=====
30.6 C
2=====
30.6 C
30.9 C
31.1 C
31.3 C
31.3 C
31.4 C
31.4 C
31.3 C
31.3 C
31.2 C
31.0 C
30.9 C
31.0 C
31.0 C
30.9 C
30.8 C
3=====
26 3 C

```

Figura 23: Teste comando para extração dos dados

Fonte: O autor (2025)

Adicionalmente, existe um comando que possibilita a leitura em tempo real da temperatura atual registrada pelo dispositivo, como ilustrado na Figura 24.

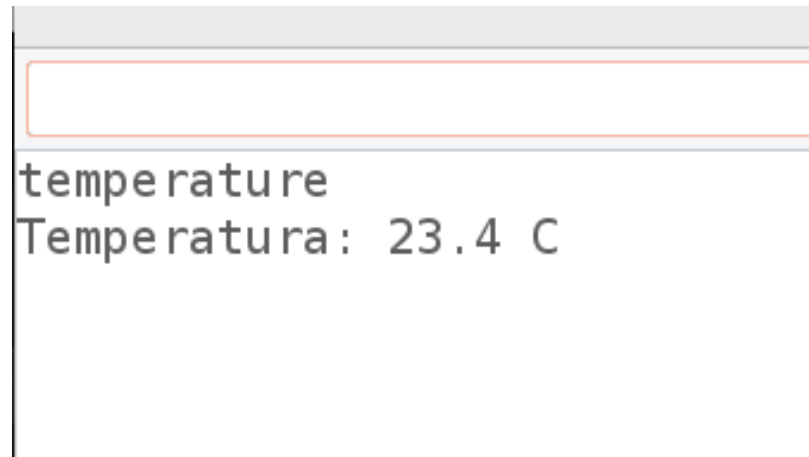


Figura 24: Teste comando para ler temperatura em tempo real

Fonte: O autor (2025)

Com isso, a interface de comandos pela UART foi validada.

5.3.3 Teste de aquisição de temperatura

O teste de aquisição dos dados foi realizado em dois dias distintos. O primeiro teste ocorreu no dia 29 de março de 2025, com uma aquisição de 2 horas e meia, das 16:30 às 19:00. O segundo teste foi realizado no dia 5 de abril de 2025, com uma aquisição de 12 horas, das 5:30 às 17:30.

Para analisar os dados coletados, foi desenvolvido um pequeno programa em Python que extrai as informações e indica quantas coletas estão disponíveis na memória. É importante observar que "coletas" refere-se ao período em que o dispositivo estava ligado, operando e realizando a aquisição das amostras de temperatura a cada 10 minutos. O programa permite selecionar uma coleta e exibir um gráfico correspondente.

A Figura 25 demonstra a interface do programa ao iniciar, mostrando a leitura das coletas e a seleção disponível. O código fonte em Python da ferramenta de extração de dados, está disponível no Apêndice C.

```
sleman@sleman:~/_git/NRF52_PROJECT$ /bin/python3 /home/sleman/_git/NRF52_PROJECT/tools/extract.py
CONECTADO /dev/ttyUSB0

Existem 7 coletas de dados disponíveis:
1: 0.00 horas
2: 2.50 horas
3: 0.00 horas
4: 0.33 horas
5: 12.00 horas
6: 0.00 horas
7: 0.83 horas

Escolha um conjunto para visualizar (0 para sair): █
```

Figura 25: Interface do pequeno programa em Python para extração dos dados

Fonte: O autor (2025)

A Figura 26 apresenta o gráfico resultante do primeiro teste de 2 horas e meia.

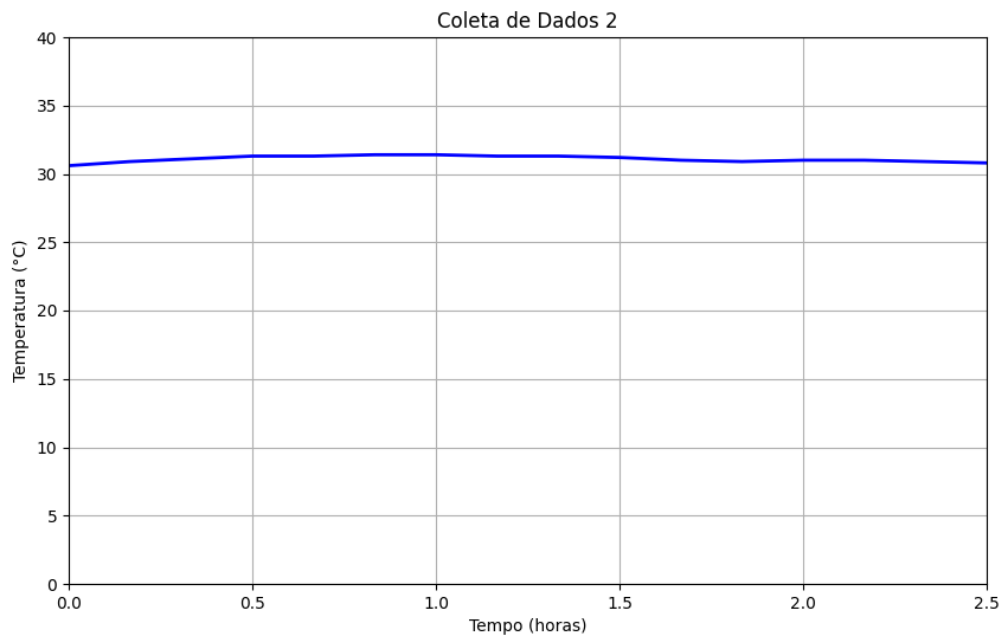


Figura 26: Gráfico da aquisição de temperatura em 2 horas e meia

Fonte: O autor (2025)

Neste gráfico, é possível observar como a temperatura variou no ambiente durante essas 2 horas e meia de aquisição. Por sua vez, a Figura 27 mostra o resultado do segundo teste, realizado ao longo de 12 horas.

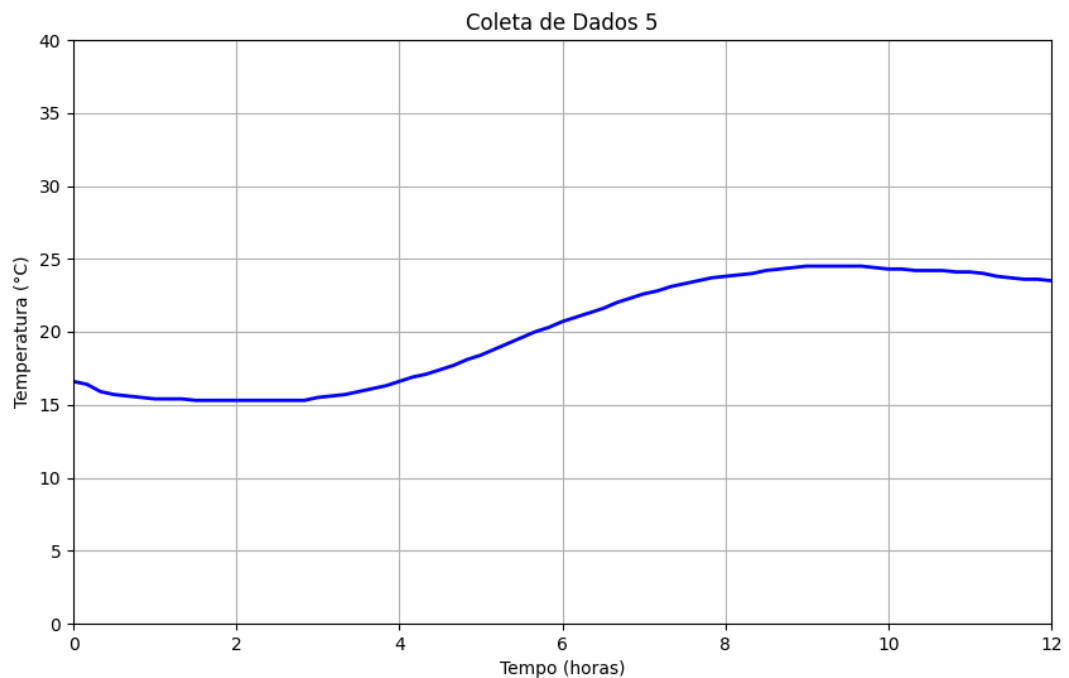


Figura 27: Gráfico da aquisição de temperatura em 12 horas

Fonte: O autor (2025)

Assim como no primeiro teste, é possível observar as variações de temperatura no ambiente em que o dispositivo foi colocado durante essas 12 horas.

É crucial destacar que, devido à ausência de um relógio integrado, o dispositivo não consegue registrar a data e a hora das aquisições. Portanto, é necessário ter ciência do número de coletas já realizadas antes de ligar o dispositivo. Alternativamente, é possível limpar o armazenamento por meio de um comando via serial, permitindo que a próxima vez que o dispositivo for ligado, registre a primeira coleta.

5.4 PLACA DE CIRCUITO IMPRESSO

Esta seção apresenta o resultado da placa de circuito impresso, que visa demonstrar a concepção do dispositivo final, dispensando o uso de uma placa Arduino e módulos auxiliares.

especificado anteriormente no esquemático. A figura 29 mostra o resultado das camadas de ligações elétricas da placa, que possui 4 camadas e uma espessura de 1,6 mm. A placa tem um tamanho compacto, medindo 36 mm por 26 mm.

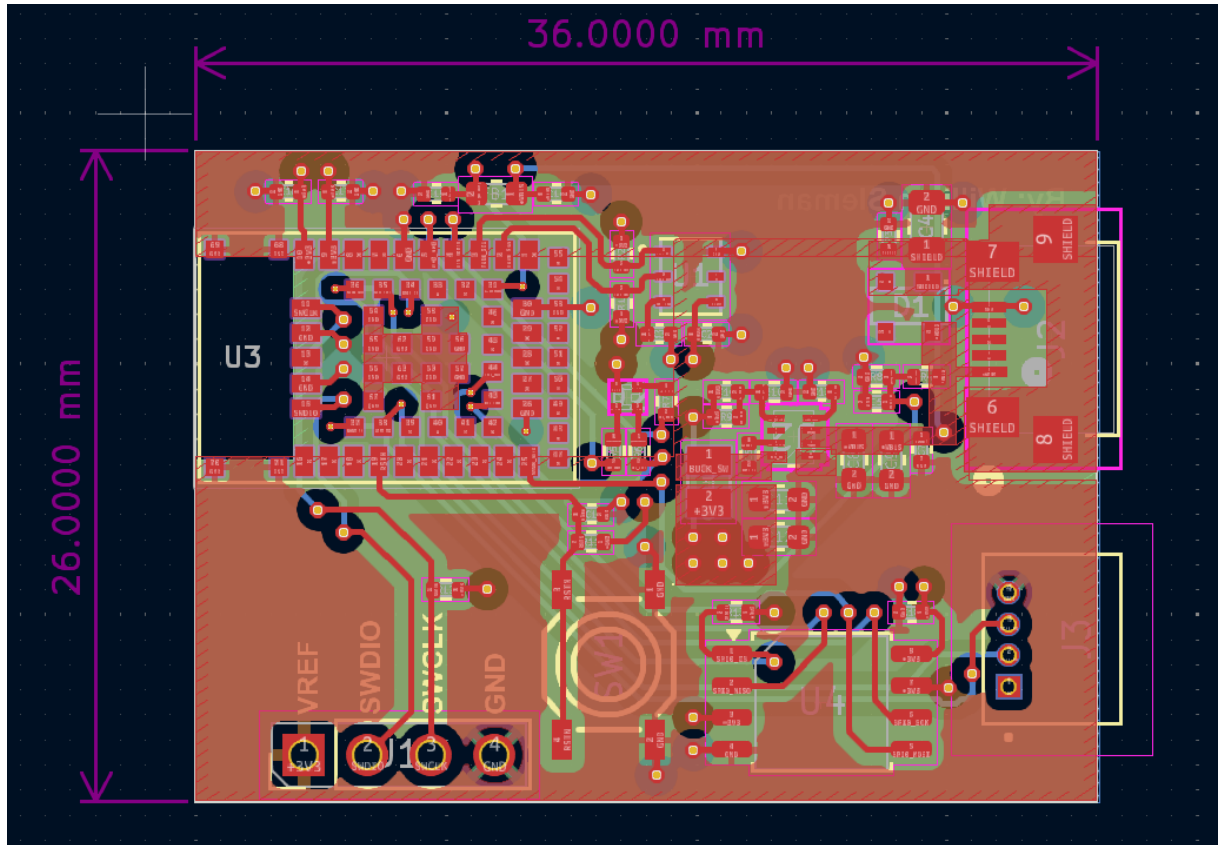


Figura 29: *Layout* da placa em visualização bidimensional

Fonte: O autor (2025)

A figura 30 exibe a visualização tridimensional da placa.

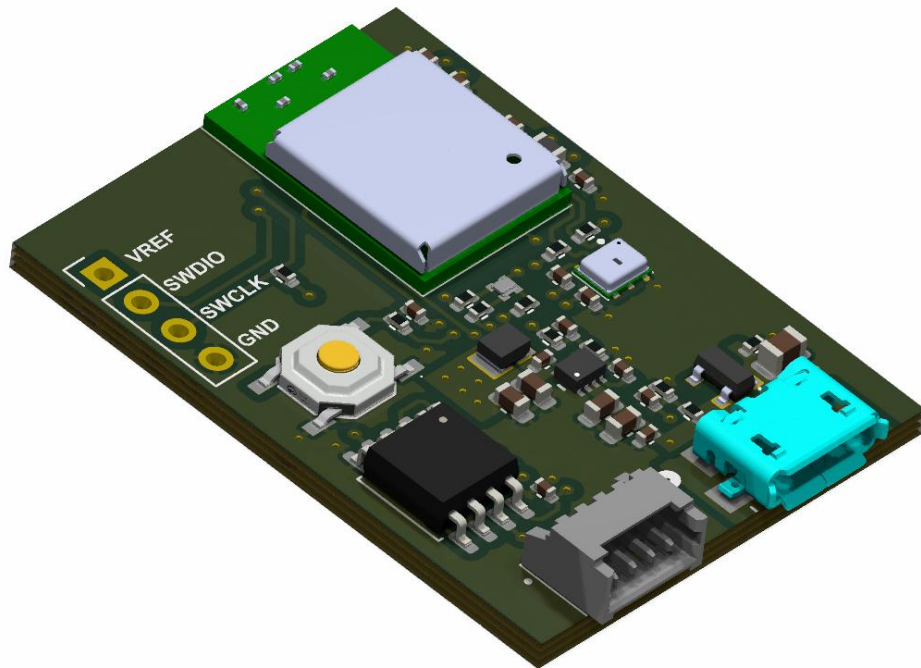


Figura 30: *Layout* da placa em visualização tridimensional

Fonte: O autor (2025)

A placa conta com três conectores: um conector USB micro B para alimentação, um conector de comunicação serial em 3,3 volts para extração de dados e um conector para gravação do firmware do dispositivo. A figura 31 esclarece a localização dos conectores na placa, além de mostrar os circuitos integrados utilizados e o módulo NINA-B306.

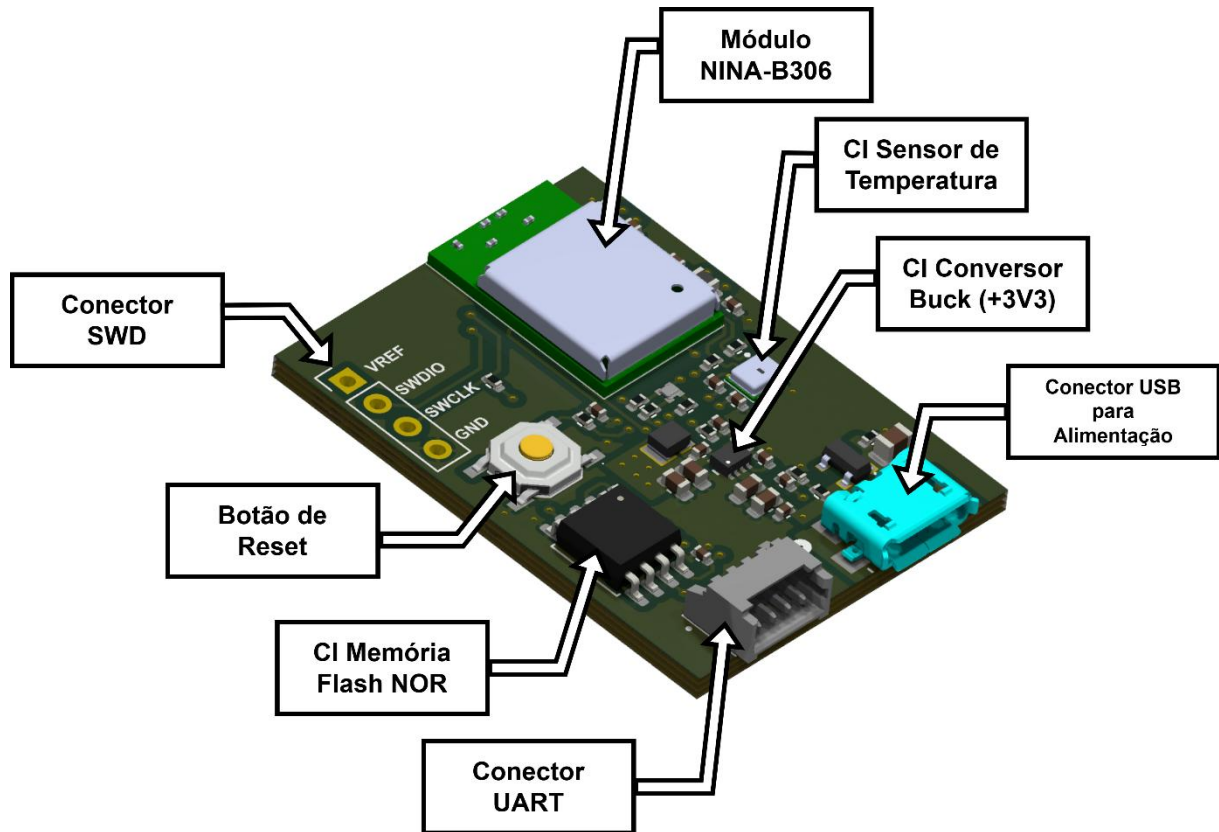


Figura 31: Layout da placa com indicações dos circuitos

Fonte: O autor (2025)

Assim, foi possível ter uma ideia de como será a placa do dispositivo. É importante ressaltar que o projeto visa apenas a visualização, e nenhuma amostra para teste funcional foi elaborada. Portanto, ajustes no *layout* da placa podem ser necessários.

6 CONCLUSÃO

O objetivo geral deste trabalho foi explorar as capacidades do *Open CPU* do módulo NINA-B306, para isso foi desenvolvido um dispositivo para aquisição de dados que registra a temperatura ambiente. Através do estudo das funcionalidades deste módulo, foi possível compreender suas funcionalidades e aplicabilidades, confirmando sua viabilidade em projetos de sistemas embarcados.

A análise e implementação da arquitetura em camadas no desenvolvimento do dispositivo possibilitou a identificação das interações entre os *drivers* de baixo e alto nível, destacando suas contribuições para o funcionamento do sistema como um todo. A separação em camadas não apenas melhorou a organização do código, mas também evidenciou a facilitação de possíveis manutenções no código.

O protótipo funcional criado em *protoboard* consolidou os objetivos específicos do trabalho, utilizando um código em linguagem C que possibilitou a aquisição e armazenamento de dados de temperatura. Os testes realizados demonstraram as capacidades do módulo NINA-B306, ressaltando o funcionamento dos seus periféricos. Este resultado prático não só válida a teoria por trás do projeto, mas fornece uma aplicação das funcionalidades discutidas ao longo do trabalho.

A finalização da placa de circuito impresso integrou todos os componentes essenciais em um *design* compacto. O que permitiu a transição do protótipo em *protoboard* para a concepção de uma aplicação prática. O desenvolvimento da PCI também trouxe experiência ao autor para futuros projetos, o que é importante em trabalhos como este.

Com a conquista desses objetivos, fica evidente que o NINA-B306, associado à arquitetura em camadas e ao desenvolvimento de software em C, representa um ótimo meio de criar dispositivos em sistemas embarcados. Essa combinação não só atendeu às necessidades do projeto, mas também lançou as bases para futuros desenvolvimentos.

Com isso, a experiência adquirida ao longo do desenvolvimento pode servir como um guia para novos projetos, demonstrando a importância de um planejamento e de uma execução de uma arquitetura no dispositivo a ser desenvolvido. O aprendizado obtido ampliou o entendimento sobre sistemas embarcados.

Por final, os objetivos propostos foram alcançados e os resultados obtidos evidenciam a viabilidade e o potencial do módulo NINA-B306 em aplicações práticas de sistemas embarcados. Este trabalho não só contribui para o avanço do conhecimento na área, mas também propõe uma metodologia para o desenvolvimento de sistemas embarcados, evitando o uso de frameworks e permitindo uma compreensão mais profunda do funcionamento interno de um microcontrolador.

REFERÊNCIAS

AFANEH, Mohammad. **Bluetooth Low Energy (BLE): A Complete Guide**. Carmel, 2022. Disponível em: <https://novelbits.io/bluetooth-low-energy-ble-complete-guide/>. Acesso em: 18 out. 2024.

ALECRIM, Emerson.; MARQUES, Ana. **O que é Bluetooth Low Energy (BLE) e quais as vantagens desse tipo de conexão?**. São Paulo, 2023. Disponível em: <https://tecnoblog.net/responde/o-que-e-bluetooth-low-energy-ble/>. Acesso em: 18 out. 2024.

ARDUINO. **Arduino Nano 33 BLE Sense Rev 2**. [S. l.], 2024. Disponível em: <https://store.arduino.cc/collections/boards-modules/products/nano-33-ble-sense-rev2>. Acesso em: 25 out. 2024.

AYINDE, Olayiwola. **Reusable Embedded Software**. Nova Deli, 2022. Disponível em: <https://dev.to/nerdhardware/reusable-embedded-software-359l>. Acesso em: 17 out. 2024.

BENINGO, Jacob. **Bootloader Design for MCUs in Embedded Systems Embedded Software Design Techniques Bootloader Design for Microcontrollers in Embedded Systems**. Linden, 2015. Disponível em: <https://www.benigo.com/insights/white-papers/bootloader-design-for-microcontrollers-in-embedded-systems/>. Acesso em: 18 out. 2024.

BENINGO, Jacob. **Reusable Firmware Development: A Practical Approach to APIs, HALs and Drivers**. New York: Springer, 2017.

BIN AFTAB, Muhammad Usama. **Building Bluetooth low energy systems : take your first steps in IoT**. Birmingham: Packt Publishing, 2017.

CATSOULIS, John. **Designing embedded hardware**. 2. ed. Sebastopol: O'Reilly Media Inc., 2005.

DAVIDSON, Scott; SHRIVER, Bruce D. **An Overview of Firmware Engineering**. Lafayette, 1978.

FIBOCOM. **Fibocom 5G FWA OpenCPU Solution**. Shenzhen, 2022. Disponível em: <https://www.fibocom.com/en/newscenter/fibocom-opencpu-solution.html?id=2776>. Acesso em: 16 out. 2024.

FTDI CHIP. **FT232R USB UART IC Datasheet Version 2.16**. Glasgow, 2021. Disponível em: https://ftdichip.com/wp-content/uploads/2020/08/DS_FT232R.pdf. Acesso em: 10 mar. 2025.

GARCIA, Fernando Deluno. **Introdução aos sistemas embarcados e microcontroladores**. São Paulo, 2018. Disponível em: <https://embarcados.com.br/sistemas-embarcados-e-microcontroladores/>. Acesso em: 1 maio 2025.

GILLIS, Alexander S.; LUTKEVICH, Ben. **What is an Embedded System? | Definition from TechTarget**. Tewksbury, 2024. Disponível em: <https://www.techtarget.com/iotagenda/definition/embedded-system>. Acesso em: 16 out. 2024.

GUPTA, Naresh. **Inside Bluetooth Low Energy Second Edition**. 2. ed. Norwood: Artech House, 2016.

HEYDON, Robin. **Bluetooth low energy : the developer's handbook**. Upper Saddle River,: Pearson Education, Inc., 2013.

HLAPISI, Nthatsi. **Exploring the Basics of Bluetooth Low Energy: A Beginners Guide To BLE**. Mafeteng, 2023. Disponível em: <https://www.allaboutcircuits.com/technical-articles/exploring-the-basics-of-bluetooth-low-energy-a-beginners-guide-to-ble/>. Acesso em: 18 out. 2024.

KOVACS, Leandro. **O que é firmware? • Aplicativos e Software • Tecnoblog**. Rio de Janeiro, 2021. Disponível em: <https://tecnoblog.net/responde/o-que-e-firmware/>. Acesso em: 17 out. 2024.

LACAMERA, Daniele. **Embedded systems architecture: discover software programming on embedded devices to build safe and secure connected systems**. Birmingham: PACKT Publishing Limited, 2023.

NOERGAARD, Tammy. **Embedded Systems Architecture: A Comprehensive Guide for Engineers and Programmers**. 2. ed. Waltham: Newnes, 2013.

NORDIC SEMICONDUCTOR. **NRF52840: Product Specification**. Trondheim, 2024. Disponível em: <https://www.nordicsemi.com/Products/nRF52840>. Acesso em: 25 out. 2024.

QUECTEL. **OpenCPU User Guide GSM/GPRS Module Series**. Shanghai, 2013. Disponível em: <https://forums.quectel.com/uploads/short-url/83RkGZzCRAMLnLF3RorJiuMBtco.pdf>. Acesso em: 1 set. 2024.

RENESAS. **High Performance Relative Humidity and Temperature Sensor HS300x Datasheet**. Koto, 2021. Disponível em: <https://www.renesas.com/en/document/dst/hs3xxx-datasheet?r=417411>. Acesso em: 9 fev. 2025.

SMARTSOC. **Embedded Systems in IoT: The Backbone of Smart Connectivity**. Bengaluru, 2024. Disponível em: <https://www.smartsocs.com/embedded-systems-in-iot/>. Acesso em: 21 abr. 2025.

TWOMEY, Jerry. **Applied embedded electronics : design essentials for robust systems**. Sebastopol: O'Reilly Media, 2024.

U-BLOX. **NINA-B3 series: Stand-alone Bluetooth 5 low energy modules**. Thalwil, 2023. Disponível em: <https://www.u-blox.com/en/product/nina-b30-series-open-cpu-0?legacy=Current#Documentation-&-resources>. Acesso em: 25 out. 2024.

WHITE, Elecia. **Making embedded systems**. 2. ed. Sebastopol: O'Reilly Media, Inc., 2024.

WINDBOND. **3V 64M-BIT SERIAL FLASH MEMORY WITH DUAL/QUAD SPI & QPI**. Taichung, 2017. Disponível em: https://www.winbond.com/hq/product/code-storage-flash-memory/serial-nor-flash/?__locale=en&partNo=W25Q64JV. Acesso em: 14 fev. 2025.

APÊNDICE A – ESQUEMÁTICO DA PLACA DE CIRCUITO IMPRESSO

PCB NINA PROJECT

2025-05-21

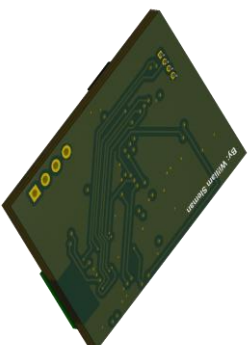
Variant: DRAFT

Page	Index	Page	Index	Page	Index	Page	Index
1	Cover Page	11		21		31	
2	Block Diagram	12		22		32	
3	Power Supply	13		23		33	
4	MCU	14		24		34	
5	Revision History	15		25		35	
6		16		26		36	
7		17		27		37	
8		18		28		38	
9		19		29		39	
10		20		30		40	

TOP VIEW



BOTTOM VIEW



NOTES

Not fitted components are marked as **X**

DRAFT - Very early stage of schematic, ignore details.
PRELIMINARY - Close to final schematic.
CHECKED - There shouldn't be any mistakes. Contact the engineer if you find any.
RELEASED - A board with this schematic has been sent to production.

DRAFT 16-MAY-2025

DESIGN CONSIDERATIONS

DESIGN NOTE:
Example text for
informational design
notes.

DESIGN NOTE:
Example text for
debug notes.

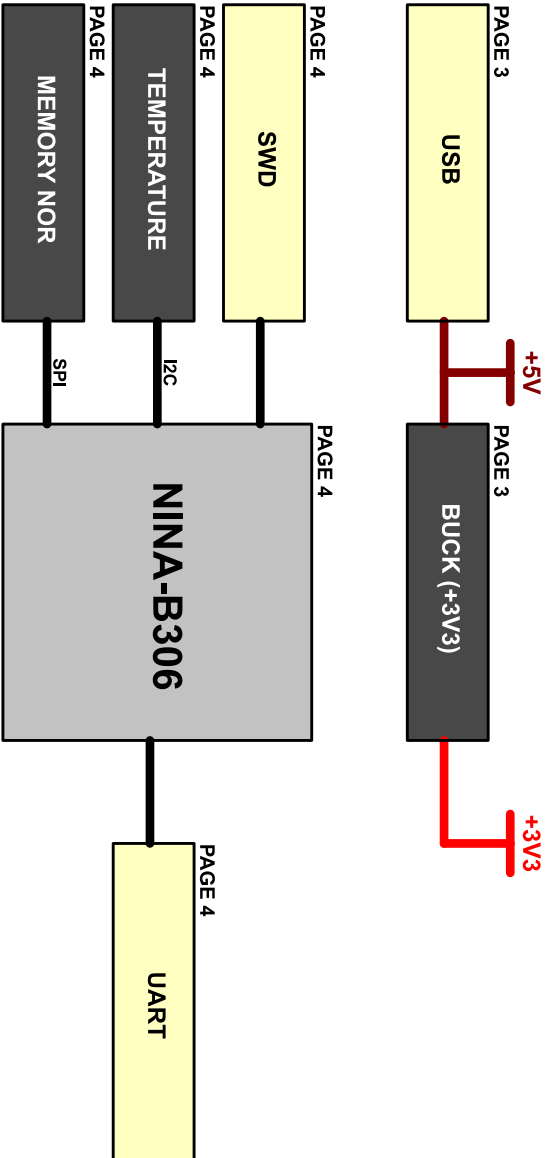
DESIGN NOTE:
Example text for
cautionary design
notes.

DESIGN NOTE:
Example text for
critical design
notes.

LAYOUT NOTE:
Example text for
critical layout
guidelines.

Author: William Simeon	
Sheet: /	
File: \\C:\PCB\lead.ch	
Title: PCB NRP52 PROJECT	
Sheet: A3	Print: 2025-05-16
PCB: ECU_9A.2	
	Rev: 1.0
	Id: 1/5

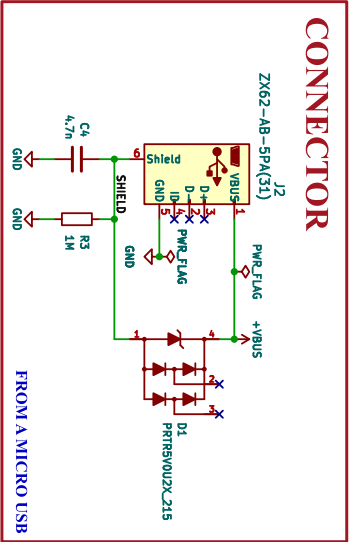
[02] - Block Diagram



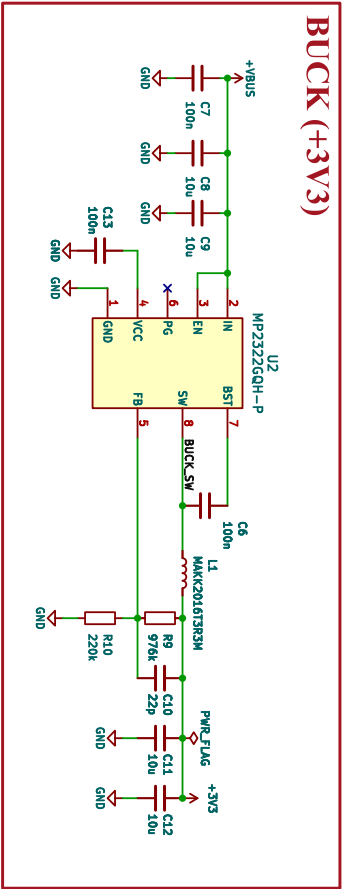
Author: William Sieman			
Sheet: /021 - Block Diagram/			
File: sheet2.sch.kicad.sch			
Title: PCB NRF52 PROJECT			
Size: A4	Date: 2025-05-16	Rev: 1.0	
KiCad E.D.A. 9.0.2		Id: 2/5	

[03] - Power Supply

CONNECTOR

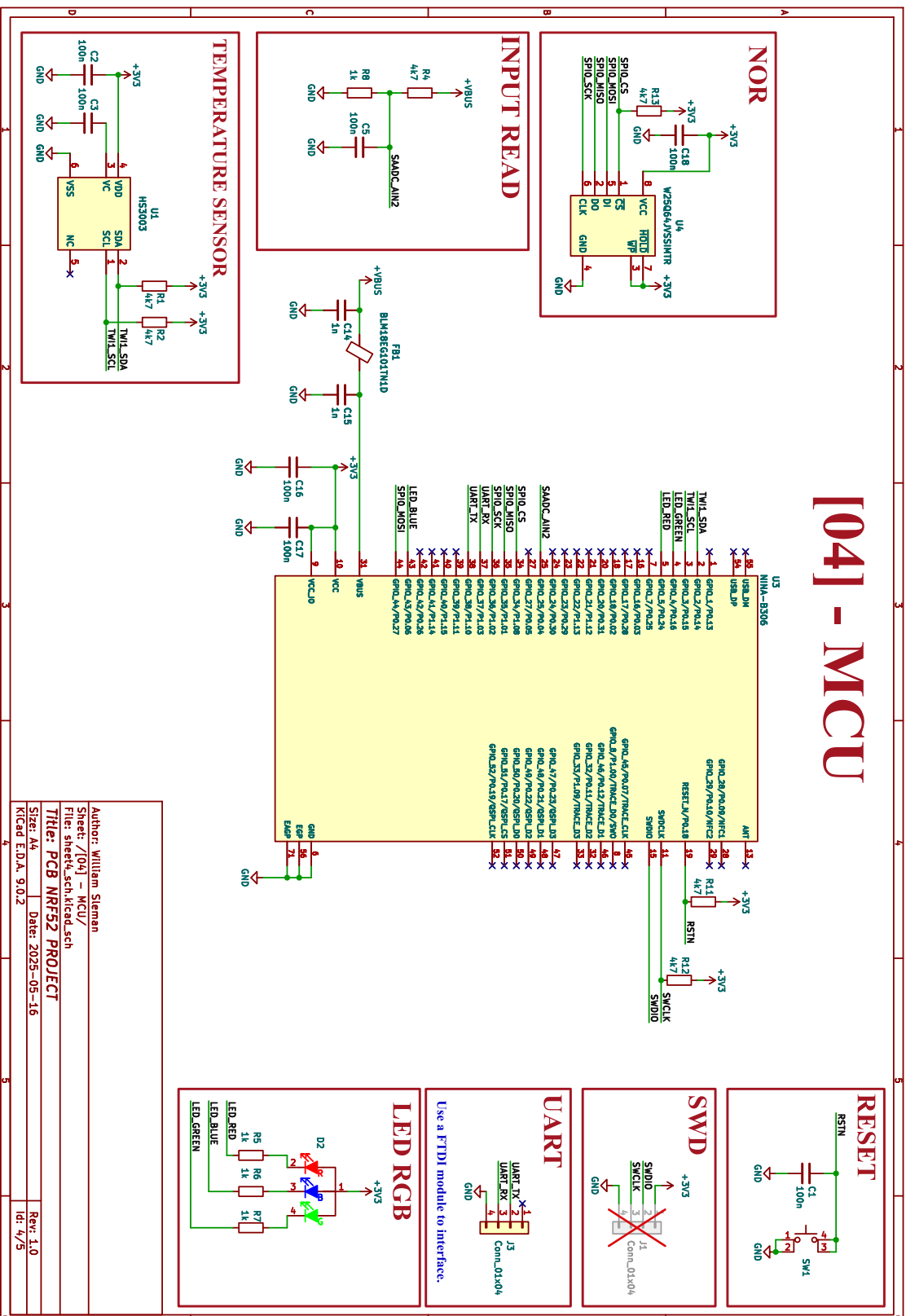


BUCK (+3V3)



Author: William Sieman	
Sheet: /[03] - Power Supply/	
File: sheet3.sch,kicad.sch	
Title: PCB NRF52 PROJECT	
Size: A4	Rev: 1.0
Date: 2025-05-16	Id: 3/5
Kicad E.D.A. 9.0.2	

[04] - MCU



APÊNDICE B – CÓDIGO FONTE EM C DA CAMADA DE APLICAÇÃO

Init_app.h

```

1  #ifndef INIT_APP_H_
2  #define INIT_APP_H_
3
4  #include <stdint.h>
5
6  #define CPU_FREQ          (64000000)
7  #define SYSTICK_FREQ      (1000) // in Hz
8
9  #define LED_BLUE_PIN      GPIO_PIN_NO_6
10 #define LED_GREEN_PIN     GPIO_PIN_NO_16
11 #define LED_RED_PIN       GPIO_PIN_NO_24
12 #define LED_BUILT_IN      GPIO_PIN_NO_13
13 #define LED1_EXT_PIN      GPIO_PIN_NO_5
14 #define BUTTON1_PORT      GPIOF1
15 #define BUTTON1_PIN       GPIO_PIN_NO_2
16
17 // UART
18 #define UART_TX_PORT      GPIOF1
19 #define UART_TX_PIN       GPIO_PIN_NO_10
20 #define UART_RX_PORT      GPIOF1
21 #define UART_RX_PIN       GPIO_PIN_NO_3
22
23 // SPI
24 #define CS_PORT            GPIOF1
25 #define CS_PIN             GPIO_PIN_NO_8
26 #define SPI_PORT_MISO      GPIOF1
27 #define SPI_PIN_MISO       GPIO_PIN_NO_1
28 #define SPI_PORT_SCK       GPIOF1
29 #define SPI_PIN_SCK        GPIO_PIN_NO_2
30 #define SPI_PORT_MOSI      GPIOP0
31 #define SPI_PIN_MOSI       GPIO_PIN_NO_27
32
33 // TWI
34 #define TWI_SDA_PORT       GPIOP0
35 #define TWI_SDA_PIN        GPIO_PIN_NO_14
36 #define TWI_SCL_PORT       GPIOP0
37 #define TWI_SCL_PIN        GPIO_PIN_NO_15
38
39
40 void system_waitStable(void);
41 void system_init(void);
42
43 uint64_t system_get_ticks(void);
44
45 #endif /* INIT_APP_H_ */

```

Init_app.c

```

1 #include "init_app.h"

```

```

2 #include "nrf52.h"
3 #include "core/ticks.h"
4
5 static volatile uint64_t ticks = 0;
6
7
8 static void system_systickInit(void);
9 static void system_setupGpio(void);
10 static void system_setupUart(void);
11 static void system_setupSpi(void);
12 static void system_setupSaadc(void);
13 static void system_setupTwi(void);
14
15 void system_waitStable(void)
16 {
17     for(uint32_t i = 0; i < 1000000; i++) __asm("NOP");
18 }
19
20 void system_init(void)
21 {
22     system_systickInit();
23     system_setupGpio();
24     system_setupUart();
25     system_setupSpi();
26     system_setupSaadc();
27     system_setupTwi();
28 }
29
30 uint64_t system_get_ticks(void)
31 {
32     return ticks;
33 }
34
35 void SysTick_Handler(void)
36 {
37     ticks_increment();
38     ticks++;
39 }
40
41 static void system_systickInit(void)
42 {
43     systick_set_frequency(SYSTICK_FREQ, CPU_FREQ);
44     systick_counter_enable();
45     systick_interrupt_enable();
46 }
47
48
49 static void system_setupGpio(void)
50 {
51     GPIO_Handle_t ledPins;
52     ledPins.pGPIOx = GPIO0;
53     ledPins.GPIO_PinConfig.GPIO_PinDir = GPIO_DIR_OUT;
54     ledPins.GPIO_PinConfig.GPIO_PinNumber = LED_GREEN_PIN;
55     ledPins.GPIO_PinConfig.GPIO_PinPuPd = GPIO_PIN_NO_PUPD;
56     ledPins.GPIO_PinConfig.GPIO_InpBuf = GPIO_INP_BUF_DISCONNECT;
57     ledPins.GPIO_PinConfig.GPIO_PinState = GPIO_PIN_SET;
58     GPIO_Init(&ledPins);

```

```

59
60     ledPins.GPIO_PinConfig.GPIO_PinNumber = LED_BLUE_PIN;
61     GPIO_Init(&ledPins);
62
63     ledPins.GPIO_PinConfig.GPIO_PinNumber = LED_RED_PIN;
64     GPIO_Init(&ledPins);
65
66     ledPins.GPIO_PinConfig.GPIO_PinNumber = LED_BUILT_IN;
67     GPIO_Init(&ledPins);
68     GPIO_WriteToOutputPin(GPIO_P0, LED_BUILT_IN, GPIO_PIN_SET);
69
70     GPIO_Handle_t inputPin;
71     inputPin.pGPIOx = BUTTON1_PORT;
72     inputPin.GPIO_PinConfig.GPIO_PinDir = GPIO_DIR_IN;
73     inputPin.GPIO_PinConfig.GPIO_PinNumber = BUTTON1_PIN;
74     inputPin.GPIO_PinConfig.GPIO_PinPuPd = GPIO_PIN_PU;
75     inputPin.GPIO_PinConfig.GPIO_InpBuf = GPIO_INP_BUF_CONNECT;
76     GPIO_Init(&inputPin);
77 }
78
79 static void system_setupUart(void)
80 {
81     UART_Handle_t UARThandle;
82     UARThandle.UART_Config.Baud = UART_STD_BAUD_115200;
83     UARThandle.UART_Config.Mode = UART_MODE_TXRX;
84
85     UARThandle.Tx.port = UART_TX_PORT;
86     UARThandle.Tx.pin = UART_TX_PIN;
87     UARThandle.Rx.port = UART_RX_PORT;
88     UARThandle.Rx.pin = UART_RX_PIN;
89
90     UART_Init(&UARThandle);
91     uart_interruptConfig(UART_INTERRUPT_RXRDY, ENABLE);
92     interrupt_config(UART_IRQ_NO, ENABLE);
93 }
94
95 static void system_setupSpi(void)
96 {
97     GPIO_Handle_t CsPin;
98     CsPin.pGPIOx = CS_PORT;
99     CsPin.GPIO_PinConfig.GPIO_PinDir = GPIO_DIR_OUT;
100    CsPin.GPIO_PinConfig.GPIO_PinNumber = CS_PIN;
101    CsPin.GPIO_PinConfig.GPIO_PinPuPd = GPIO_PIN_NO_PUPD;
102    CsPin.GPIO_PinConfig.GPIO_PinState = GPIO_PIN_SET;
103    CsPin.GPIO_PinConfig.GPIO_InpBuf = GPIO_INP_BUF_DISCONNECT;
104    GPIO_Init(&CsPin);
105
106    SPI_Handle_t SPIHandle;
107    SPIHandle.pSPIx = SPI0;
108    SPIHandle.SPI_Config.CPHA = SPI_CPHA_LOW;
109    SPIHandle.SPI_Config.CPOL = SPI_CPOL_LOW;
110    SPIHandle.SPI_Config.Frequency = SPI_FREQUENCY_K125;
111
112    SPIHandle.MISO.port = SPI_PORT_MISO;
113    SPIHandle.MOSI.port = SPI_PORT_MOSI;
114    SPIHandle.SCK.port = SPI_PORT_SCK;
115

```

```

116     SPIHandle.MISO.pin = SPI_PIN_MISO;
117     SPIHandle.MOSI.pin = SPI_PIN_MOSI;
118     SPIHandle.SCK.pin  = SPI_PIN_SCK;
119     SPI_Init(&SPIHandle);
120 }
121
122 static void system_setupSaadc(void)
123 {
124     // A2 -> P0.4
125     SAADC_Handle_t SAADCHandle;
126     SAADCHandle.CHANNEL = SAADC_CHANNEL_0;
127     SAADCHandle.RESN    = SAADC_RES_BYPASS;
128     SAADCHandle.RESP    = SAADC_RES_BYPASS;
129     SAADCHandle.GAIN    = SAADC_GAIN_1_4;
130     SAADCHandle.REFSEL  = SAADC_REFSEL_VDD_1_4;
131     SAADCHandle.MODE    = SAADC_MODE_SE;
132     SAADCHandle.TACQ    = SAADC_TACQ_15US;
133     SAADCHandle.RESOLUTION = SAADC_RESOLUTION_12BIT;
134
135     SAADCHandle.PSELP    = SAADC_PSEL_AIN2;
136     SAADCHandle.PSELN    = SAADC_PSEL_NC;
137
138     saadc_init(&SAADCHandle);
139 }
140
141 static void system_setupTwi(void)
142 {
143     TWI_Handle_t TWIHandle;
144     TWIHandle.config.FREQUENCY = TWI_FREQUENCY_K100;
145     TWIHandle.pTWIx = TWI1;
146
147     TWIHandle.SCL.port = TWI_SCL_PORT;
148     TWIHandle.SCL.pin = TWI_SCL_PIN;
149
150     TWIHandle.SDA.port = TWI_SDA_PORT;
151     TWIHandle.SDA.pin = TWI_SDA_PIN;
152
153     TWI_Init(&TWIHandle);
154 }
155
156 // retarget printf
157 extern int __io_putchar(int ch)
158 {
159     uart_write_byte((uint8_t)ch);
160     return ch;
161 }

```

Main.c

```
1  #include <stdio.h>
2  #include "init_app.h"
3  #include "core/cli.h"
4  #include "core/storage.h"
5  #include "core/ticks.h"
6  #include "core/status.h"
7
8
9  int main(void)
10 {
11     system_waitStable();
12     system_init();
13
14
15     system_waitStable();
16     printf("Init system ok...\n\r");
17
18     storage_updateCount();
19     storage_temperatureAppend();
20
21     uint64_t start_time = ticks_get();
22     while(1)
23     {
24         while(status_read() != STATUS_OK);
25         command_update();
26
27
28         // append every 10 minutes
29         if((system_get_ticks() - start_time) >= 600000)
30         {
31             storage_temperatureAppend();
32             start_time = ticks_get();
33         }
34     }
35 }
```

APÊNDICE C – CÓDIGO FONTE EM PYTHON DA FERRAMENTA DE EXTRAÇÃO DE DADOS

Serial_app.py

```

1 import serial
2 import serial.tools.list_ports
3
4 def byte_to_hex(byte_str):
5     return ''.join( [ "%02X" % ord( x ) for x in byte_str ] ).strip()
6
7 def HexToByte(hexStr):
8     bytes = []
9     hexStr = ''.join( hexStr.split(" ") )
10
11     for i in range(0, len(hexStr), 2):
12         bytes.append( chr( int( hexStr[i:i+2], 16 ) ) )
13     return ''.join( bytes )
14
15 class serialApp ():
16     def __init__(self):
17         self.serial_port = serial.Serial()
18         self.baudrate = [9600,115200]
19         self.portlist = []
20         self.serial_port.timeout = 1
21
22     def update_port(self): #updates de portas seriais
23         self.portlist =[port.device for port in serial.tools.list_ports.comports()]
24         print (self.portlist)
25
26     def connect_serial(self):
27         try:
28             self.serial_port.open()
29             print("CONECTADO",self.serial_port.port)
30             return True
31         except:
32             print("Houve um erro")
33             return False
34
35     def close_serial(self):
36         self.serial_port.close()
37         print("DESCONECTADO", self.serial_port.port)
38
39     def read_serial(self):
40         data = self.serial_port.readline()
41         return data
42
43     def send_serial (self,data):
44         if(self.serial_port.isOpen()):
45             message = str(data) + "\r\n"
46             self.serial_port.write(message.encode())
47
48     def clear_serial(self):
49         self.serial_port.reset_input_buffer()
50         self.serial_port.reset_output_buffer()

```

```

51
52     def get_count(self):
53         self.clear_serial()
54         self.send_serial("count")
55         message = self.read_serial()
56         message = self.read_serial()
57
58         count = int((message.decode()).replace("Count: ", "").replace("\n", ""))
59         return count
60
61     def get_coletas(self):
62         self.send_serial("storage extract")
63
64         values = [[]]
65
66         message = self.read_serial()
67         message = self.read_serial()
68         message = self.read_serial()
69         message = self.read_serial()
70         idx = 0
71         while(message != b''):
72             if(b'=' in message):
73                 idx += 1
74                 values.append([])
75             else:
76                 value = float((message.decode()).replace(" C\n", ""))
77                 values[idx].append(value)
78
79             message = self.read_serial()
80
81         return values

```

Coletas.py

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def coletas_show(matriz):
5     """
6     Mostra quantos coletas de dados existem e suas características
7     """
8     print(f"\nExistem {len(matriz)} coletas de dados disponíveis:")
9     for i, conjunto in enumerate(matriz):
10         minutos = (len(conjunto) - 1)*10
11         print(f"{i+1}: {minutos/60:.2f} horas")
12
13 def coletas_plot(matriz, indice):
14     """
15     Plota a coleta de dados selecionado com marcação temporal
16     """
17     conjunto = matriz[indice]
18
19     # Criar array de tempo (intervalos de 10 minutos)
20     tempo = np.arange(len(conjunto)) * (10/60)

```

```

21
22     plt.figure(figsize=(10, 6))
23     plt.plot(tempo, conjunto, 'b', linewidth=2)
24     plt.grid(True)
25     plt.title(f'Coleta de Dados {indice+1}')
26     plt.xlabel('Tempo (horas)')
27     plt.ylabel('Temperatura (°C)')
28     plt.ylim(0,40)
29     plt.xlim(0, tempo[-1])
30     plt.show()

```

Extract.py

```

1 from serial_app import serialApp
2 from coletas import *
3
4 ser = serialApp()
5
6 matriz_dados = [[]]
7
8
9 def main():
10     #ser.update_port()
11     ser.serial_port.port = '/dev/ttyUSB0'
12     ser.serial_port.baudrate = 115200
13     ser.connect_serial()
14
15     matriz_dados = ser.get_coletas()
16
17     coletas_show(matriz_dados)
18
19     # User choice
20     while True:
21         try:
22             escolha = int(input("\nEscolha um conjunto para visualizar (0 para sair): "))
23             if escolha == 0:
24                 ser.close_serial()
25                 break
26             elif 1 <= escolha <= len(matriz_dados):
27                 coletas_plot(matriz_dados, escolha-1)
28             else:
29                 print("Escolha inválida!")
30         except ValueError:
31             print("Por favor, digite um número válido!")
32
33 if __name__ == "__main__":
34     main()

```

APÊNDICE D – ACESSO AO CÓDIGO FONTE COMPLETO

Para acessar o código fonte completo utilizado neste trabalho, este está disponível no repositório público do GitHub. O leitor pode acessar a versão mais atualizada do código a partir do link abaixo:

Repositório GitHub: https://www.github.com/slemanz/NINA_PROJECT

Dentro do repositório, o código está organizado na pasta "FW". Este diretório contém a implementação de todo o trabalho discutido nesta monografia, permitindo uma análise do desenvolvimento e da estrutura do projeto.